What is Matplotlib

- **Matplotlib is a 2D and 3D graph plotting python library that serves as a visualization utility.**
- **Matplotlib was created by John D.Hunter.**
- **Matplotlib is open source and we can use it freely.**
- **Matplotlib is mostly written in Python, a few segments are written in C, Objective-c and Javascript for platform compatibility.**
- **Using matplotlib, you can draw a different graph like below:**
  - **Line plot**
  - **Histogram**
  - **Bar Chart**
  - **Pie Chart**
  - **Scatterplot**
  - **Area Plot**
  - **Error Charts**
  - **Power Spectra**
  - **etc.**

Why use Matplotlib

- **As a machine learning engineer or data scientist, data analysis is a part of model development. Looking numeric data, you can't get correct insights. So the best way to plot a graph using that data and take a decision for further process. Data visualization is process take after the data cleaning.**
- **It produce high quality graph.**
- **It also support animation.**

Download and install Matplotlib

The matplotlib installation process is very simple. If you are using Anaconda Navigator, then no need to install matplotlib. If you are using Python idle, PyCharm, Sublime Text, etc then follow the below steps.
Open the command prompt or terminal and enter

In [ ]:
```
pip install matplotlib
```

and press Enter Key to download and install matplotlib. The pip is a package, which installs the matplotlib python library.

Import Matplotlib

Once Matplotlib is installed, import it in your applications by adding the import module statement:

In [1]:
```
import matplotlib
```

Now Matplotlib is imported and ready to use:

Pyplot

Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias:

In [2]:
```
import matplotlib.pyplot as plt
```

Now the Pyplot package can be referred to as plt.

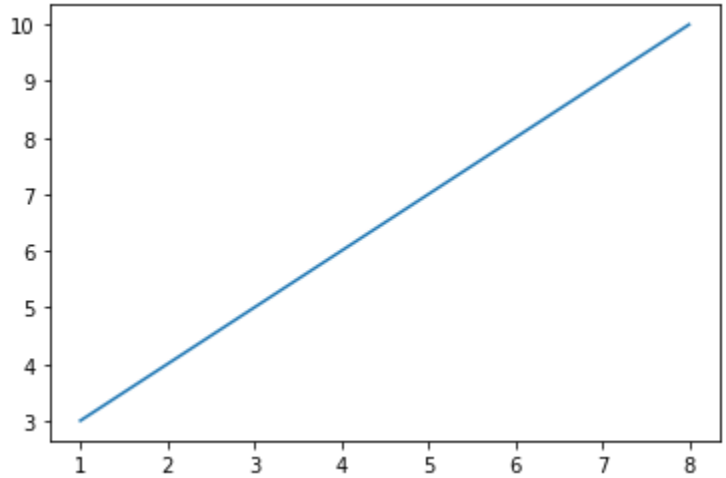
Matplotlib Plotting

Plotting x and y points

- **The plot() function is used to draw points (markers) in a diagram.**
- **By default, the plot() function draws a line from point to point.**
- **The function takes parameters for specifying points in the diagram.**
- **Parameter 1 is an array containing the points on the x-axis.**
- **Parameter 2 is an array containing the points on the y-axis.**
- **If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the plot function.**

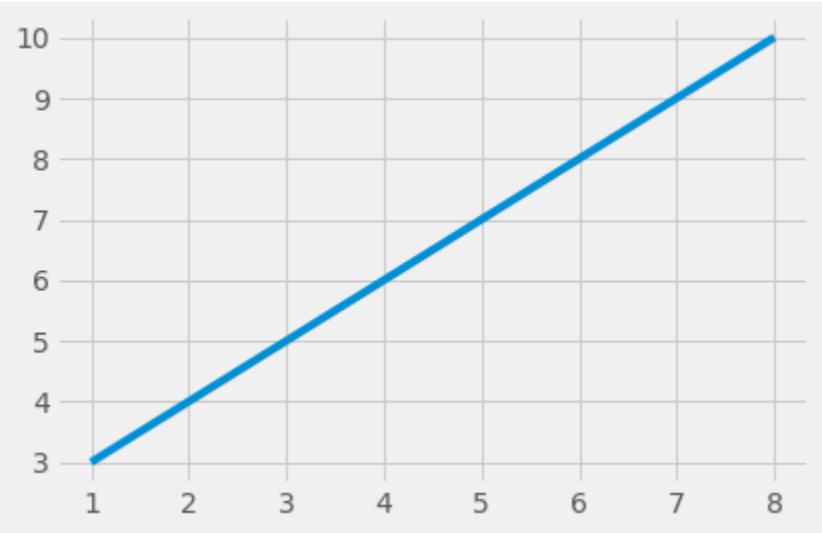
EXAMPLE: Draw a line in a diagram from position (1, 3) to position (8, 10):

In [3]:
```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints)
plt.show()
```



The X-axis is the horizontal axis.
The Y-axis is the vertical axis.

In [15]:
```
# You can directly plot the graph without using the NumPy array.
import matplotlib.pyplot as plt

xpoints =[1, 8]
ypoints =[3, 10]

plt.plot(xpoints, ypoints)
plt.show()
```



Plotting Without Line

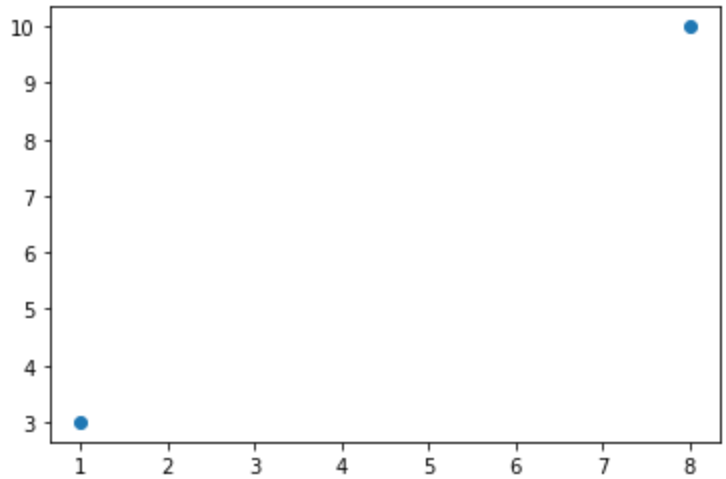
To plot only the markers, you can use shortcut string notation parameter 'o', which means 'rings'.

EXAMPLE: Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

In [4]:
```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1,8])
ypoints = np.array([3,10])
```

```
plt.plot(xpoints, ypoints, 'o')
plt.show
```

Out[4]: <function matplotlib.pyplot.show(close=None, block=None)>



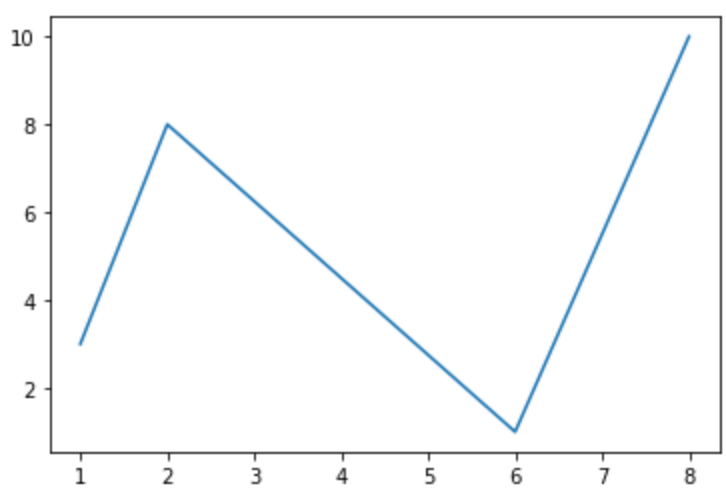
In jupyter it is not required to use show() function. But in other IDLE it is required to use show() function to show the graph.

Multiple Points

You can plot as many points as you like, just make sure you have the same number of points in both axis.

Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

In [5]:
```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1,2,6,8])
ypoints = np.array([3,8,1,10])

plt.plot(xpoints, ypoints)
plt.show()
```

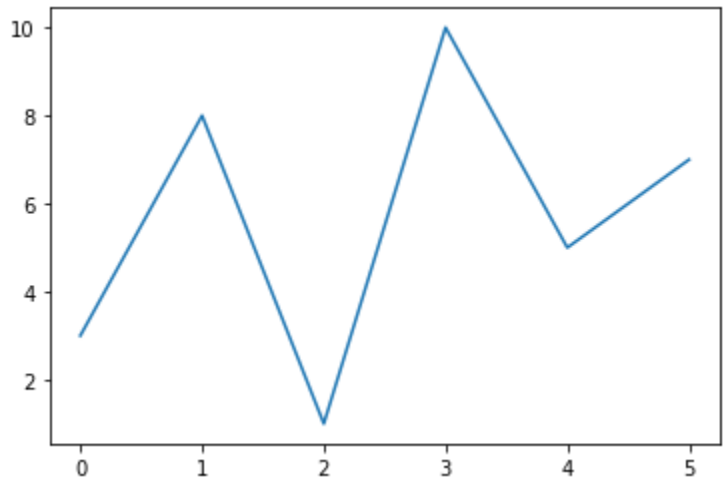


Default X-Points

- **If we do not specify the points in the x-axis, they will get the default values 0, 1, 2, 3, (etc. depending on the length of the y-points)**

- **So, if we take the same example as above, and leave out the x-points, the diagram will look like this:**

In [6]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3,8,1,10,5,7])

plt.plot(ypoints)
plt.show()
```

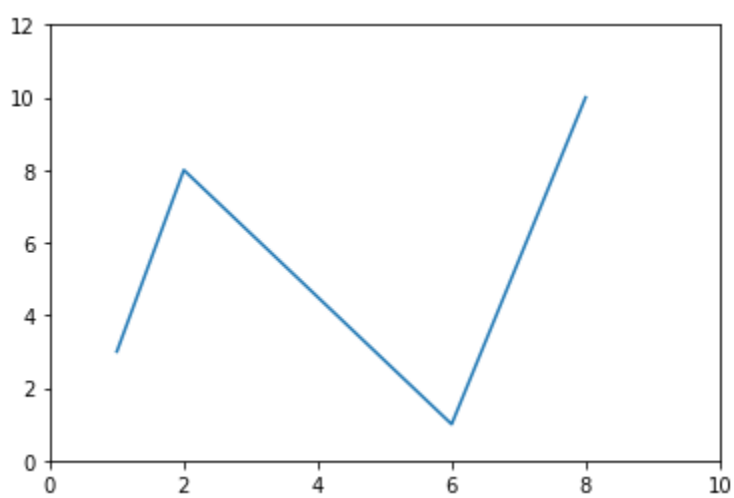


The x-points in the example above is [0, 1, 2, 3, 4, 5].

Set axis starting point

You can set the starting point of X and y axis by using axis() function.

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([1,2,6,8])
y = np.array([3,8,1,10])

plt.axis([0,10,0,12])   # First two is for X-axis and Others are for Y-axis
plt.plot(x,y)
plt.show()
```



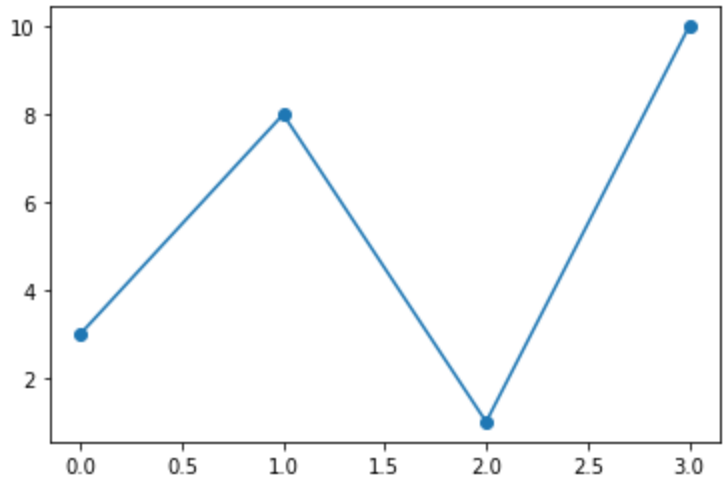
Matplotlib Markers

Markers

You can use the keyword argument marker to emphasize each point with a specified marker.

EXAMPLE: Mark each point with a circle:

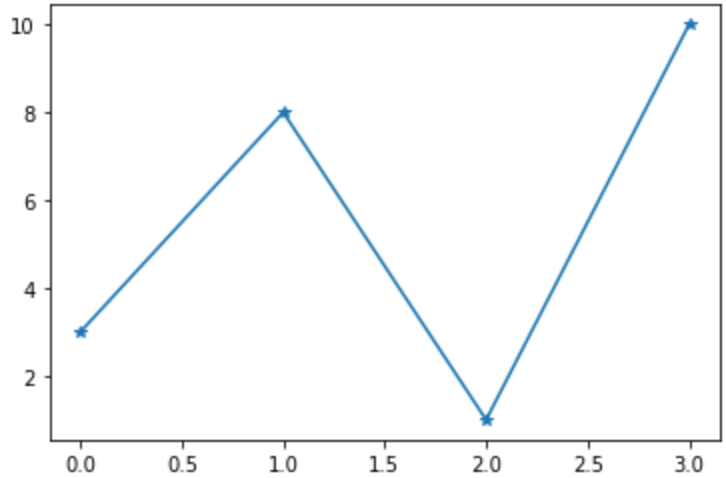
In [8]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3,8,1,10])

plt.plot(ypoints, marker = 'o')
plt.show()
```



EXAMPLE: Mark each point with a star:

In [9]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3,8,1,10])

plt.plot(ypoints, marker = '*')
plt.show
```

Out[9]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```

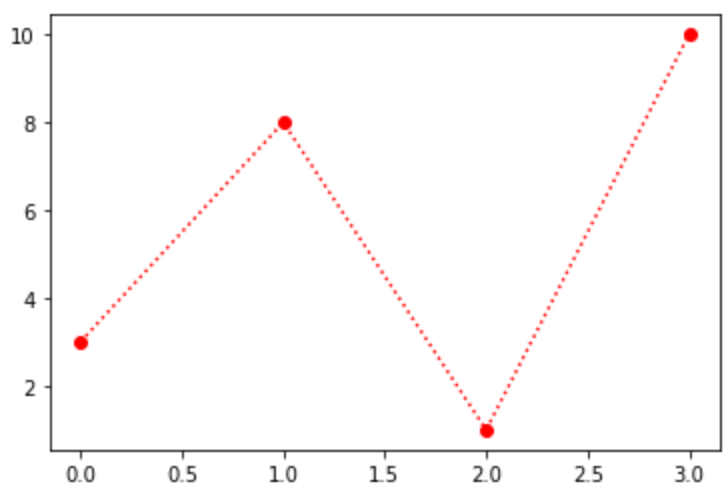


Format Strings fmt

- **You can use also use the shortcut string notation parameter to specify the marker.**
- **This parameter is also called fmt.**
- **Syntax:marker|line|color**

EXAMPLE: Mark each point with a circle:

In [10]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3,8,1,10])

plt.plot(ypoints, 'o:r')
plt.show()
```



o:r

o = Marker value ('o','*' etc)

: = Line value

r = Color reference

Let's understand them properly:

Line reference

Line Syntax	Description
'-'	Solid line
':'	Dotted line
'--'	Dashed line
'-.'	Dashed/dotted line

Note: If you leave out the line value in the fmt parameter, no line will be plottet.

Color Reference

The short color value can be one of the following:

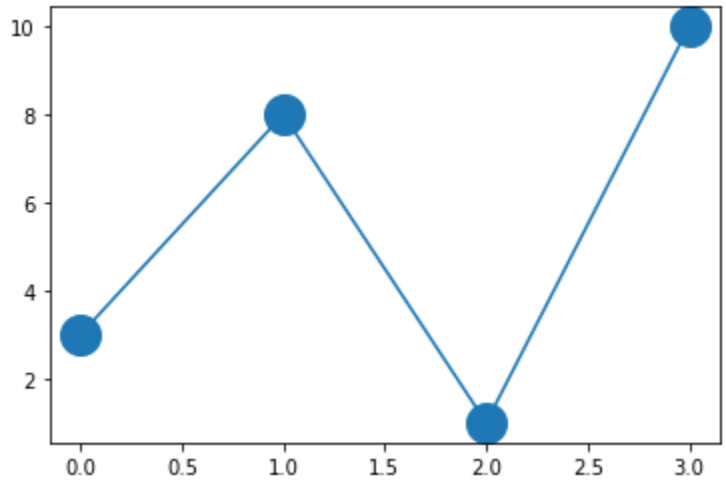
Color Syntax	Description
'r'	Red
'g'	Green
'b'	Blue
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

Marker Size

You can use the keyword argument markersize or the shorter version ms to set the size of the markers.
EXAMPLE:Set the size of the markers to 20:

In [2]:
```
import matplotlib.pyplot as plt
import numpy as np
```

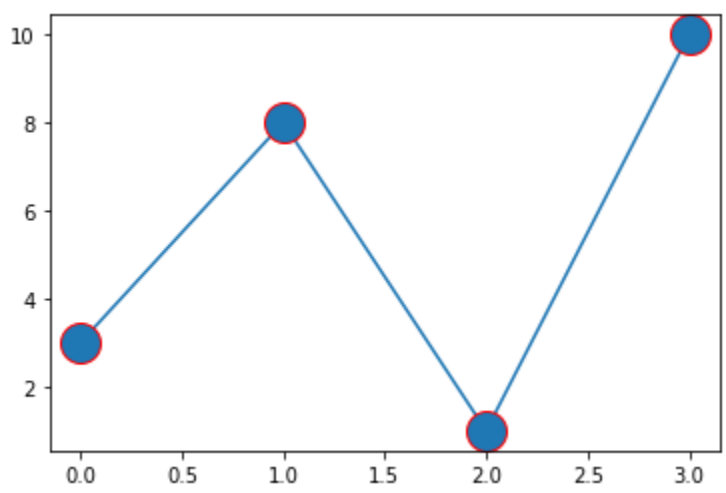
```
ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o', ms = 20)
plt.show()
```



Marker Color

You can use the keyword argument markeredgecolor or the shorter mec to set the color of the edge of the markers:
Example:Set the EDGE color to red:

In [12]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o', ms = 20, mec = 'r')
plt.show()
```

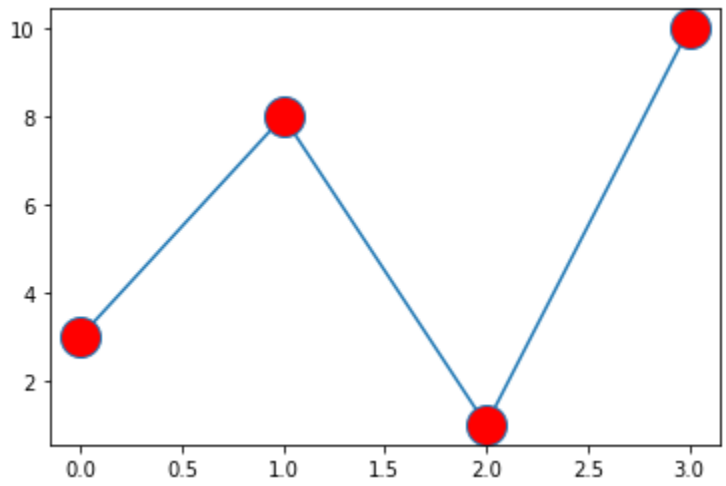


Marker face Color

You can use the keyword argument markerfaceecolor or the shorter mfc to set the color inside the edge of the markers:

In [13]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])
```

```
plt.plot(ypoints, marker = 'o', ms = 20, mfc = 'r')
plt.show()
```

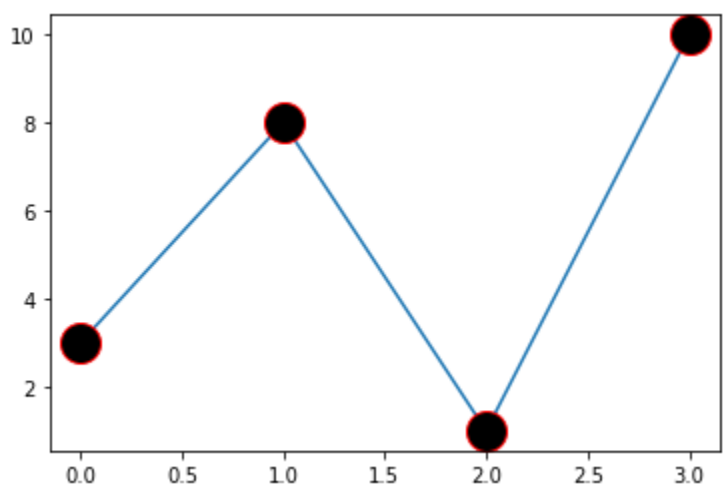


Use both the mec and mfc arguments to color of the entire marker:

EXAMPLE: Set the color of both the edge and the face:

In [14]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3,8,1,10])

plt.plot(ypoints,marker = 'o', ms = 20, mec = 'r', mfc = 'k')
```

Out[14]:
```
[<matplotlib.lines.Line2D at 0x220dd149550>]
```



Matplotlib Line:

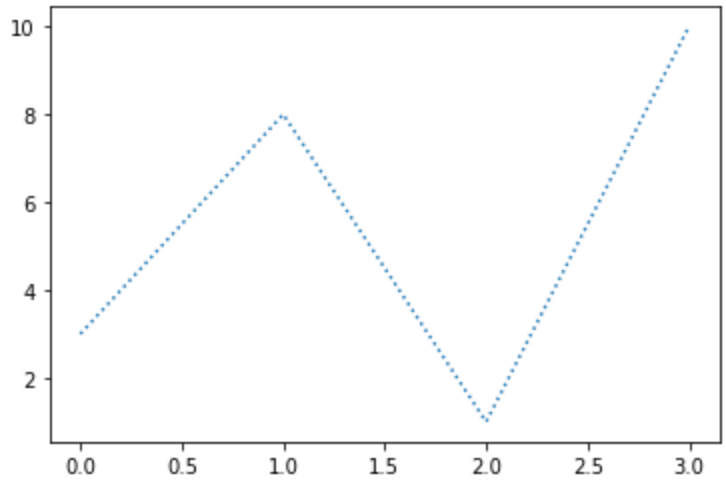
Linestyle

You can use the keyword argument linestyle, or shorter ls, to change the style of the plotted line:

EXAMPLE: Use a dotted line:

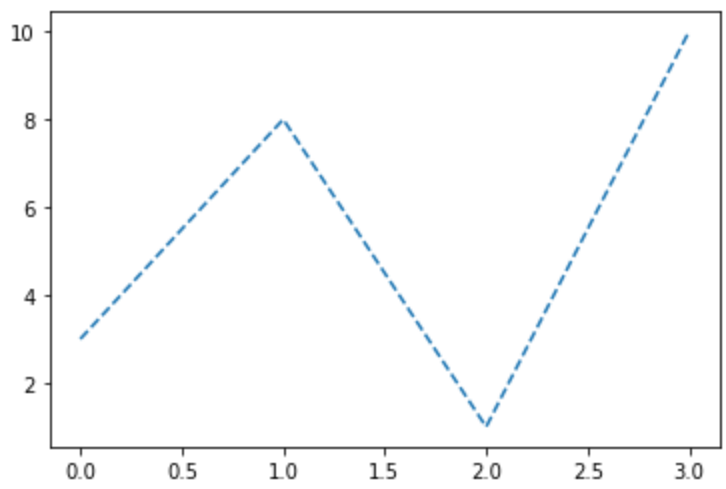
In [15]:
```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3,8,1,10])

plt.plot(ypoints, linestyle = 'dotted')
plt.show()
```



EXAMPLE: Use a dashed line

In [4]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3,8,1,10])

plt.plot(ypoints, linestyle = 'dashed')
plt.show()
```

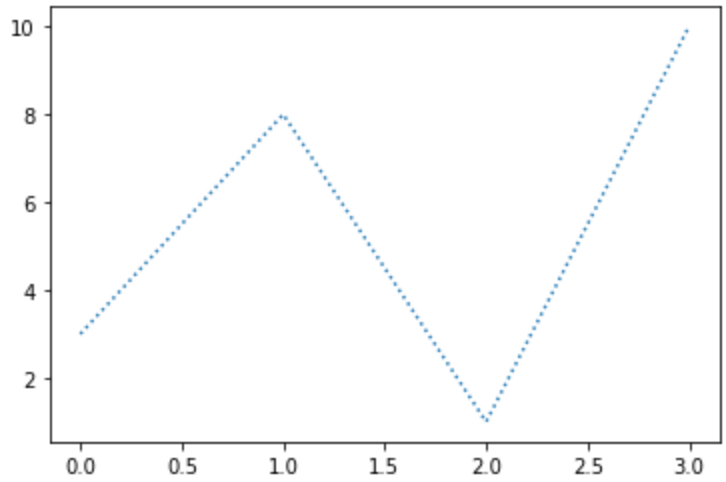


Shorter Syntax

- **The line style can be written in a shorter syntax:**
- **linestyle can be written as ls.**
- **dotted can be written as :.**
- **dashed can be written as --.**

In [17]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3,8,1,10])
```

```
plt.plot(ypoints, ls = ':')

plt.show()
```



Line Styles:

You can choose any of these styles

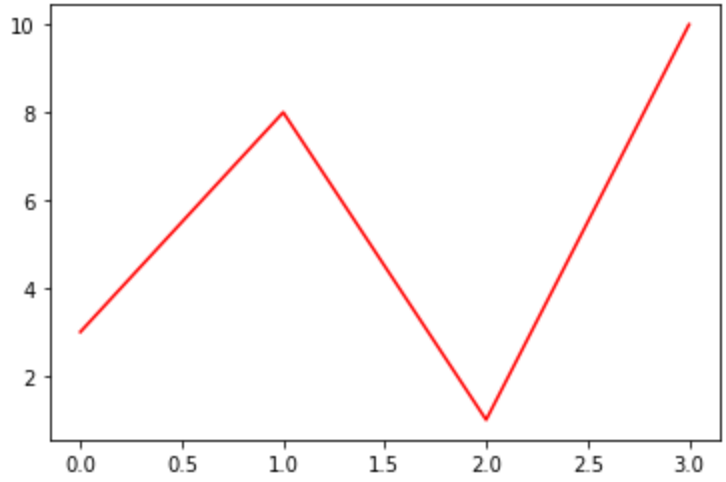
Style	Or
'solid' (default)	'-'
'dotted'	':'
'dashed'	'--'
'dashdot'	'-.'
'None'	'' or ' '

Line Color

You can use the keyword argument color or the shorter c to set the color of the line:

Example:Set the line color to red

In [18]:
```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3,8,1,10])

plt.plot(ypoints, c = 'r')
plt.show()
```

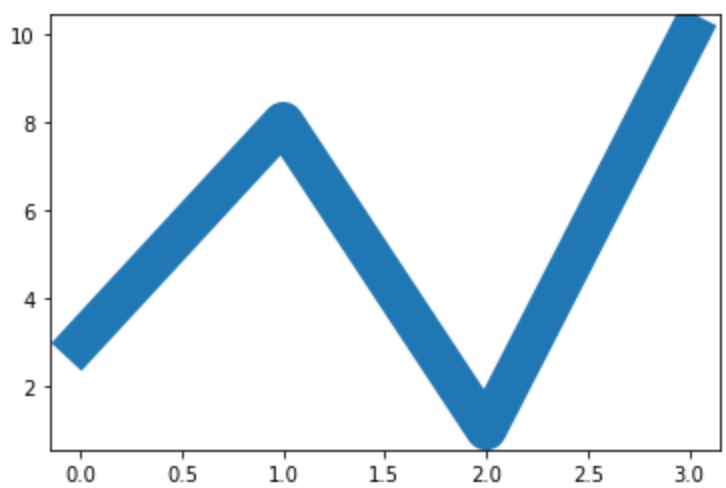


Line Width

- **You can use the keyword argument linewidth or the shorter lw to change the width of the line.**
- **The value is a floating number, in points:**

EXAMPLE: Plot with a 20.5pt wide line

In [19]:
```
import matplotlib.pyplot as plt

ypoints = np.array([3,8,1,10])

plt.plot(ypoints, linewidth = '20.5')
plt.show()
```



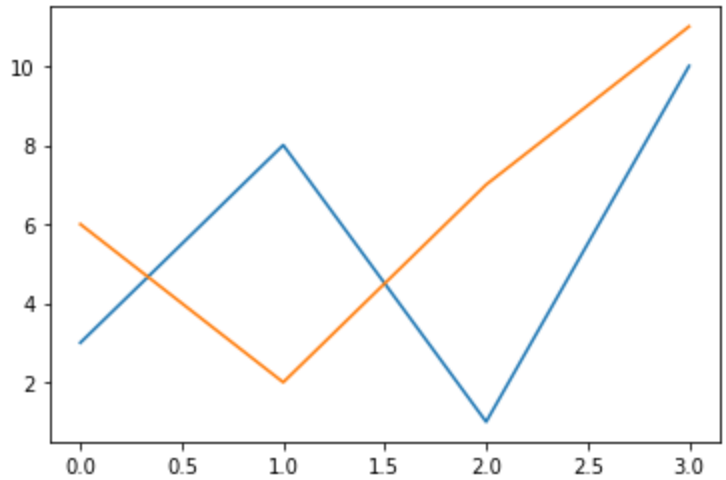
Multiple Lines

You can plot as many lines as you like by simply adding more plt.plot() functions:

Example: Draw two lines by specifying a plt.plot() function for each line

In [20]:
```
import matplotlib.pyplot as plt
import numpy as np

y1 = np.array([3,8,1,10])
y2 = np.array([6,2,7,11])
```

```
plt.plot(y1)
plt.plot(y2)

plt.show()
```



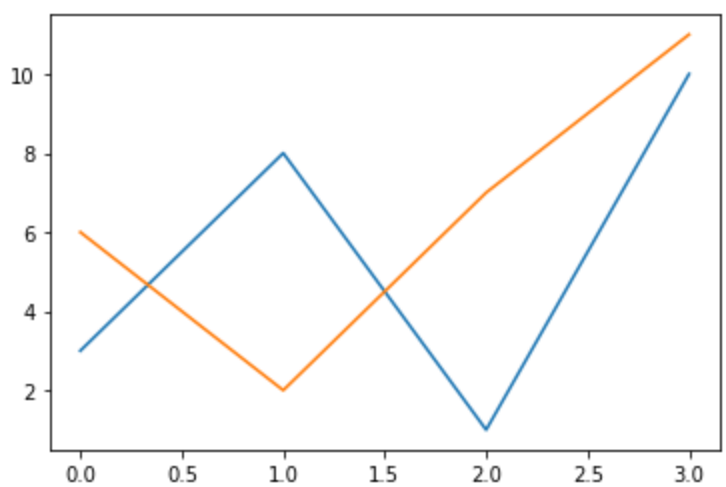
You can also plot many lines by adding the points for the x- and y-axis for each line in the same plt.plot() function.

(In the examples above we only specified the points on the y-axis, meaning that the points on the x-axis got the the default values (0, 1, 2, 3).)

The x- and y- values come in pairs:

EXAMPLE: Draw two lines by specifiyng the x- and y-point values for both lines

In [21]:
```
import matplotlib.pyplot as plt
import numpy as np

x1 = np.array([0,1,2,3])
x2 = np.array([3,8,1,10])
x3 = np.array([0,1,2,3])
x4 = np.array([6,2,7,11])

plt.plot(x1,x2,x3,x4)
plt.show()
```



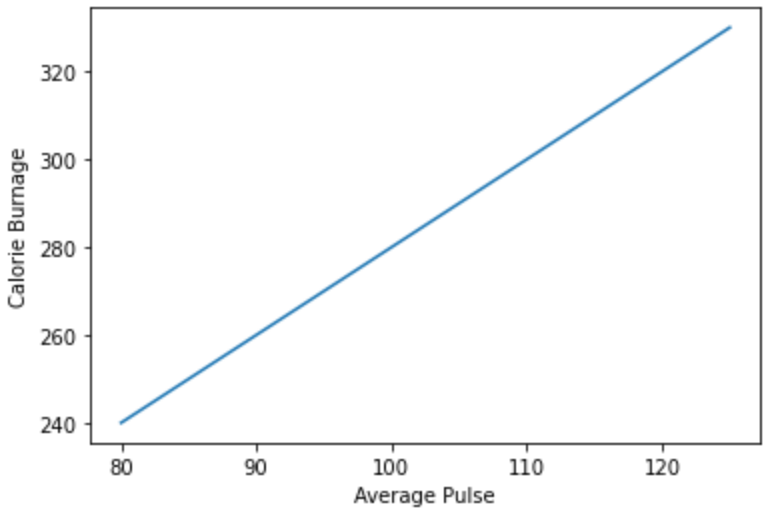
Matplotlib Labels and Title:

Create Labels for a Plot

With Pyplot, you can use the xlabel() and ylabel() functions to set a label for the x- and y-axis.

EXAMPLE: Add labels to the x- and y-axis

In [22]:
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80,85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x,y)

plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

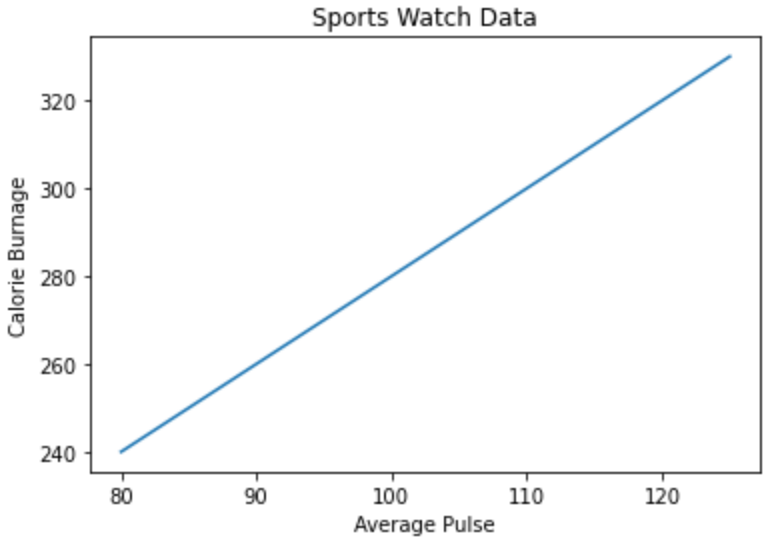


Create a Title for a Plot

With Pyplot, you can use the title() function to set a title for the plot.

EXAMPLE: Add a plot title and labels for the x- and y-axis

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x,y)

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

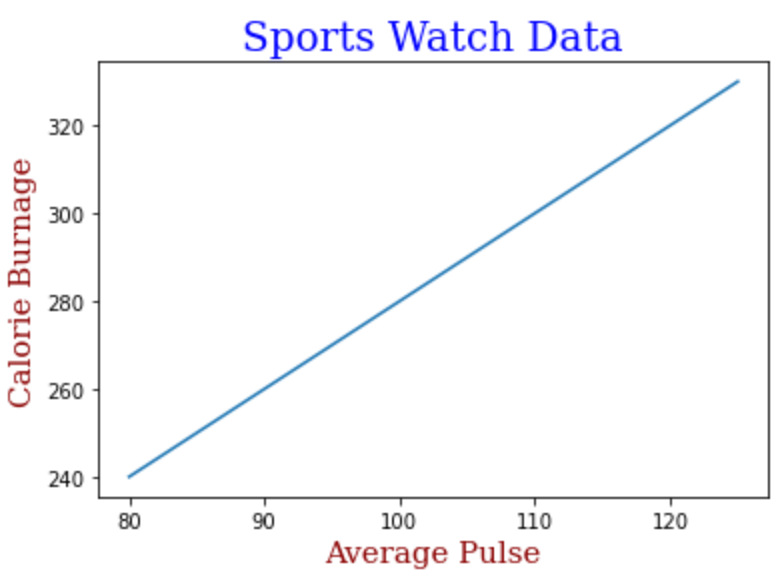


Set Font Properties for Title and Labels

You can use the fontdict parameter in xlabel(), ylabel(), and title()</font>, title() to set font properties for the title and labels.

Example: Set font properties for the title and labels

In [24]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

font1 = {'family':'serif','color':'blue','size':20}
font2 = {'family':'serif','color':'darkred','size':15}

plt.title("Sports Watch Data", fontdict = font1)
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burnage", fontdict = font2)

plt.plot(x,y)
plt.show()
```



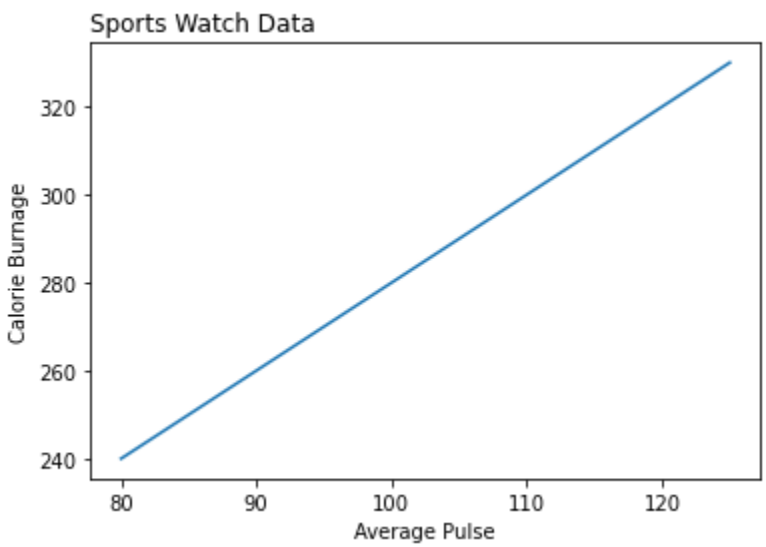
Position the Title

You can use the </font>, loc parameter in </font>, title() to position the title.

arguments:{'left', 'right', and 'center'. Default value is 'center'}

Example: Position the title to the left

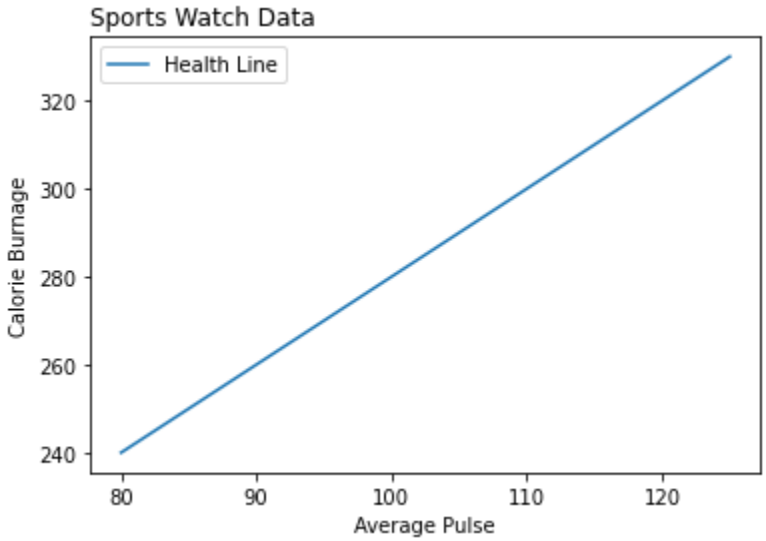
In [25]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data", loc = 'left')
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x,y)
plt.show()
```



Line Name

You can give the Name to your line by using legend function.

In [26]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data", loc = 'left')
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x,y)

plt.legend(["Health Line"])
plt.show()
```



NOTE: You can change the position of the legend by giving arguments loc

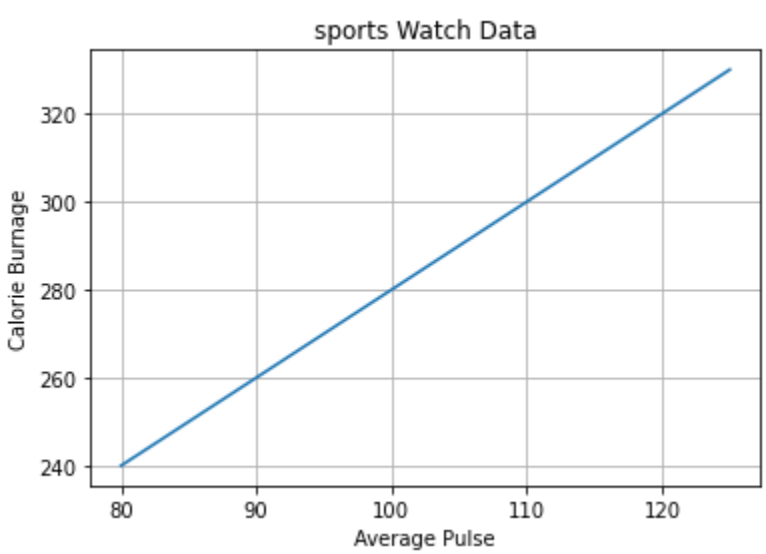
Matplotlib Adding Grid Lines:

Add Grid Lines to a Plot

With Pyplot, you can use the grid() function to add grid lines to the plot.

Example:Add grid lines to the plot

In [27]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x,y)

plt.grid()

plt.show()
```



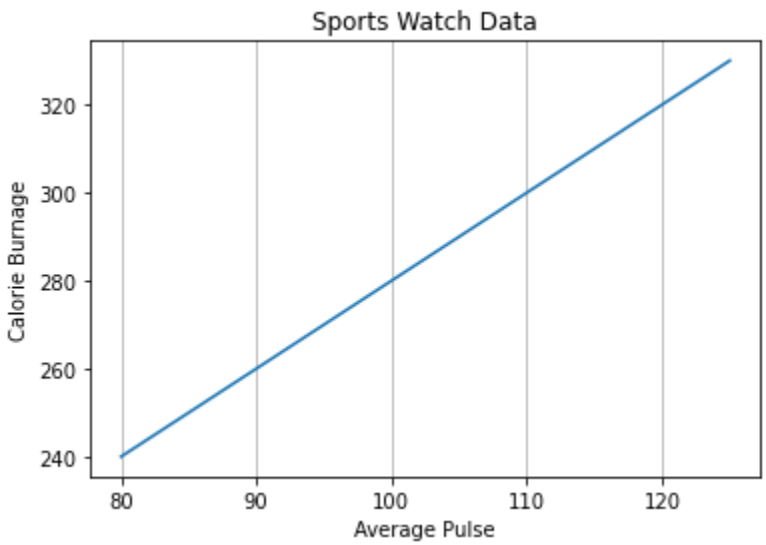
Specify Which Grid Lines to Display

You can use the axis parameter in the grid() function to specify which grid lines to display.

Legal values are: 'x', 'y', and 'both'. Default value is 'both'.

EXAMPLE: Display only grid lines for the x-axis

In [28]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x,y)

plt.grid(axis = 'x')

plt.show()
```



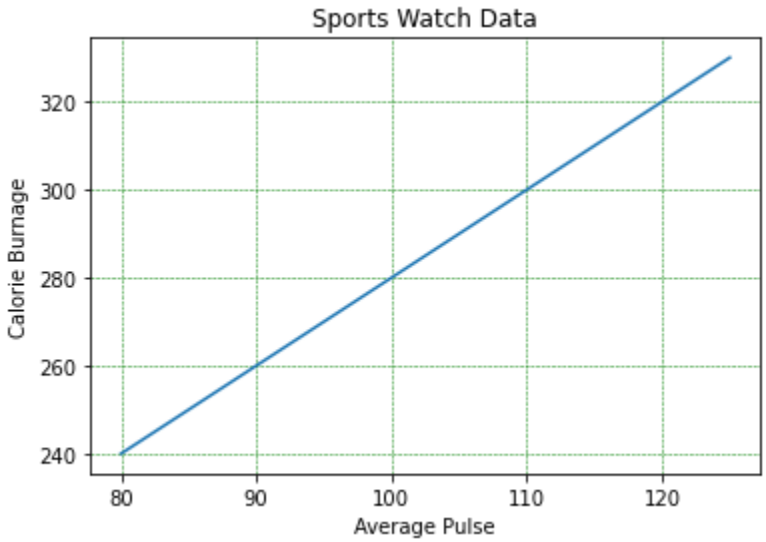
Set Line Properties for the Grid

You can also set the line properties of the grid, like this: grid(color = 'color', linestyle = 'linestyle', linewidth = number).

Example: Set the line properties of the grid

In [29]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")
```

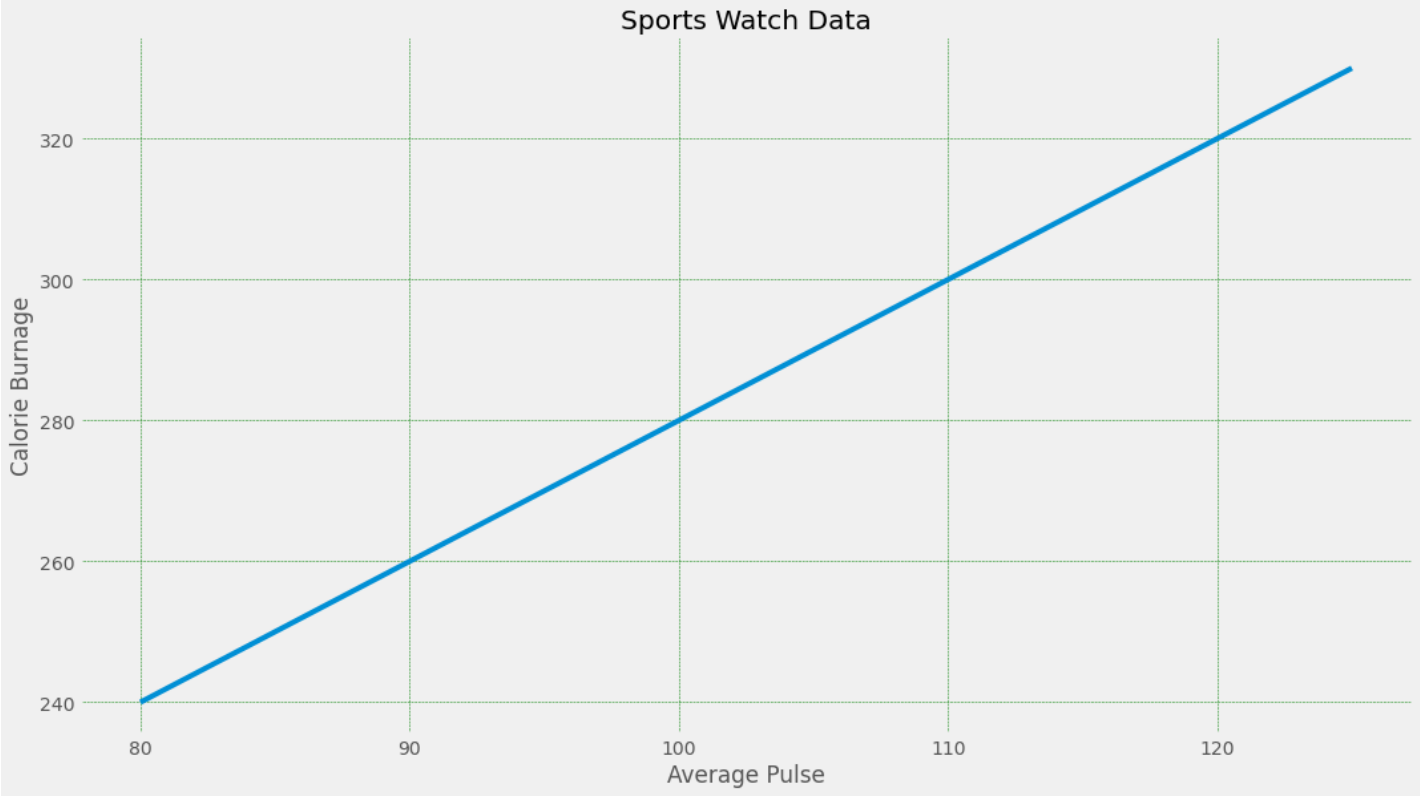
```
plt.plot(x,y)

plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)

plt.show()
```



Change the size of the figure

You can change the size of the figure by using figure() function.

In [26]:
```
import matplotlib.pyplot as plt
import numpy as np

# Lets change the size of the figure
plt.figure(figsize = (16,9))

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x,y)

plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)

plt.show()
```



Always use the figure() function at starting.

Style:

With Matplotlib, we have styles which serve a very similar purpose to Matplotlib graphs as CSS (cascading style sheet) pages serve for HTML.
The idea of a style page is to write your customization to a style file, and then, to use those changes and apply them to your graph, all you do is import style and then use that specific style.

First, you will need to import the style module from matplotlib:

In [30]:
```
from matplotlib import style
```

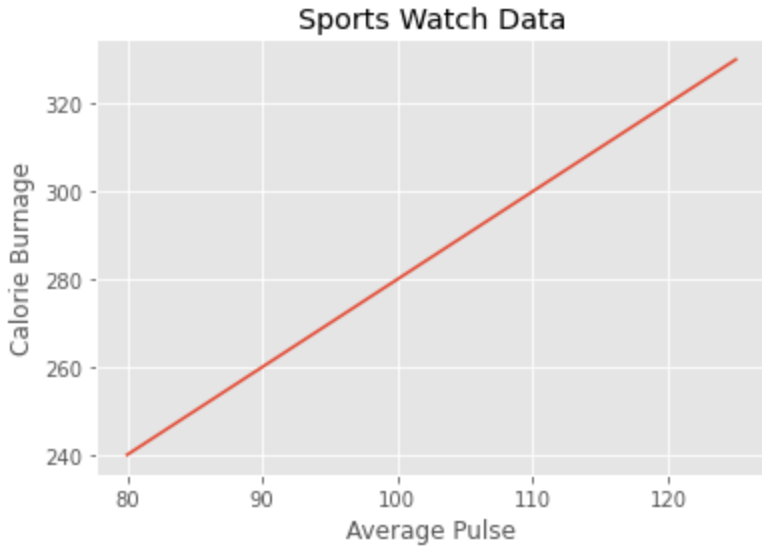
Next, we specifiy what style we want to use. Matplotlib comes with a few styles already.Like

In [31]:
```
style.use('ggplot')
```

Let's see on graph

In [6]:
```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import style

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

style.use("ggplot")
```

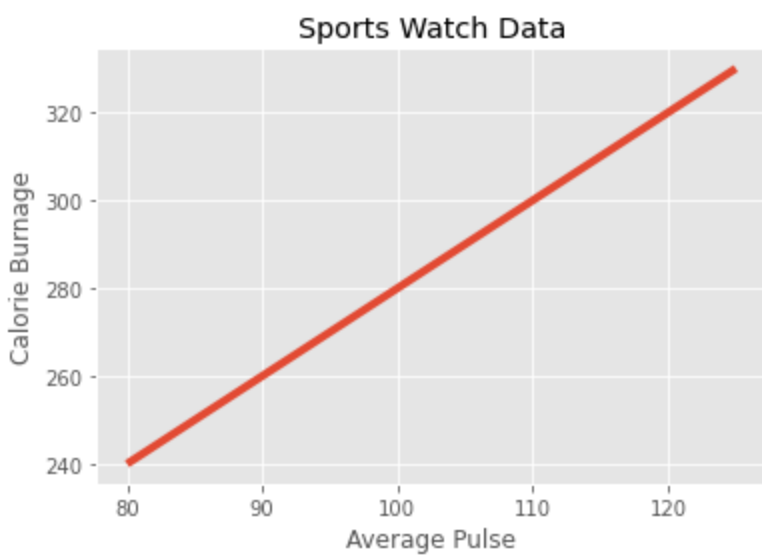
```
plt.plot(x,y)
plt.show()
```



Immediately, we can tell the font is different, besides the title, the color of labels are grey, and the axis background is a light grey. We also notice the grid is actually a solid white line.

Another example of a style is from fivethirtyeight :

In [7]:
```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import style

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

style.use("fivethirtyeight")

plt.plot(x,y)
plt.show()
```



You can see all of the available styles you currently have by doing:

In [34]:
```
print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', 'bmh', 'classic', 'dark_background', 'fast', 'f
ivethirtyeight', 'ggplot', 'grayscale', 'seaborn', 'seaborn-bright', 'seaborn-colorblind',
'seaborn-dark', 'seaborn-dark-palette', 'seaborn-darkgrid', 'seaborn-deep', 'seaborn-mute
d', 'seaborn-notebook', 'seaborn-paper', 'seaborn-pastel', 'seaborn-poster', 'seaborn-tal
k', 'seaborn-ticks', 'seaborn-white', 'seaborn-whitegrid', 'tableau-colorblind10']
```

You can choose any and use them.

Matplotlib Subplots:

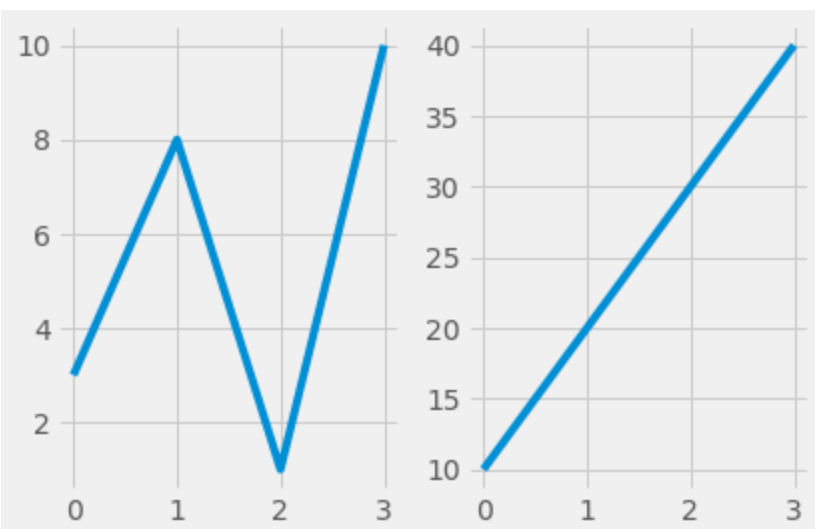
Display Multiple Plots

With the subplots() function you can draw multiple plots in one figure:

EXAMPLE: Draw 2 plots

In [35]:

```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```



The subplots() Function

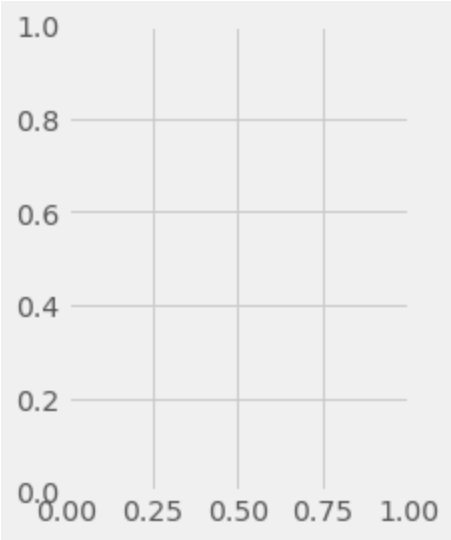
The subplots() function takes three arguments that describes the layout of the figure.

The layout is organized in rows and columns, which are represented by the first and second argument.

The third argument represents the index of the current plot.

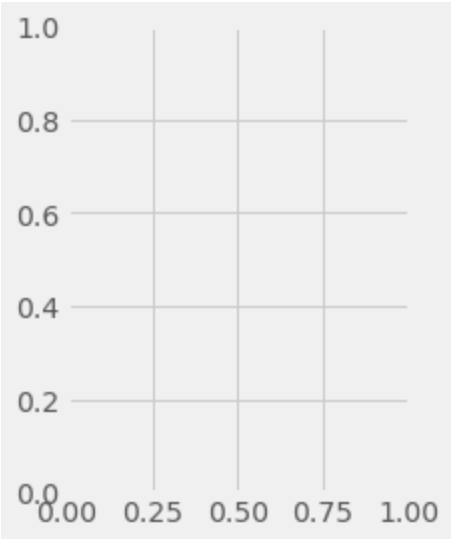
In [8]:
```
plt.subplot(1, 2, 1)
#the figure has 1 row, 2 columns, and this plot is the first plot.
```

Out[8]:
```
<AxesSubplot:>
```



In [37]:
```
plt.subplot(1, 2, 2)
#the figure has 1 row, 2 columns, and this plot is the second plot.
```

Out[37]:
```
<AxesSubplot:>
```

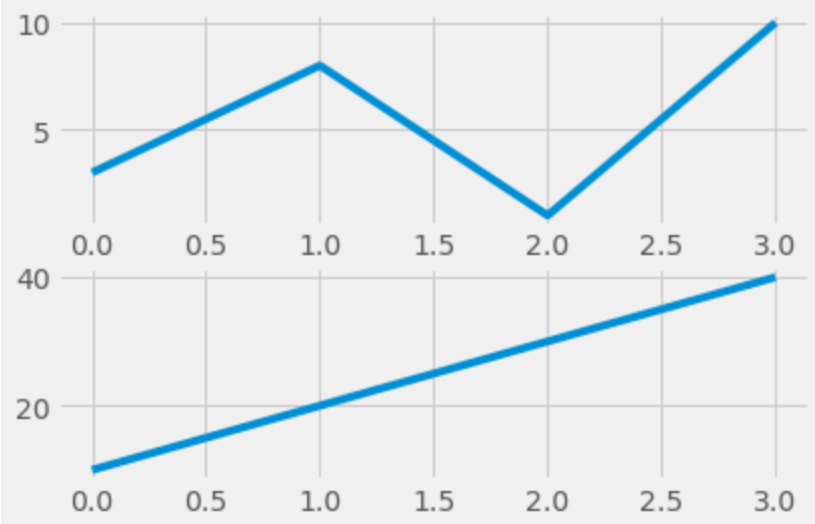


So, if we want a figure with 2 rows an 1 column (meaning that the two plots will be displayed on top of each other instead of side-by-side), we can write the syntax like this:

EXAMPLE: Draw 2 plots on top of each other

In [38]:
```
import matplotlib.pyplot as plt
import numpy as np

#plot 1
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2,1,1)
plt.plot(x,y)
```

```
#plo 2
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2,1,2)
plt.plot(x,y)

plt.show()
```



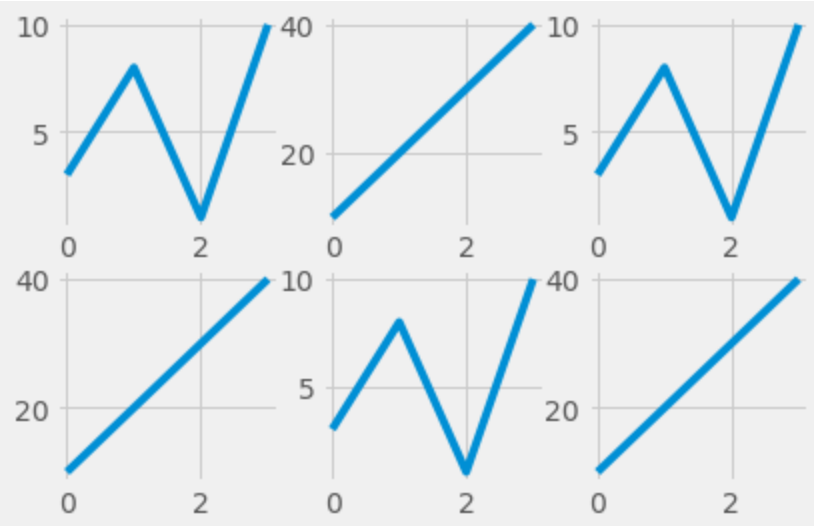
You can draw as many plots you like on one figure, just descibe the number of rows, columns, and the index of the plot.

EXAMPLE: Draw 6 plots

In [39]:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 3, 1)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 2)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 3, 3)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 4)
plt.plot(x,y)

x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(2, 3, 5)
plt.plot(x,y)
```

```
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(2, 3, 6)
plt.plot(x,y)

plt.show()
```



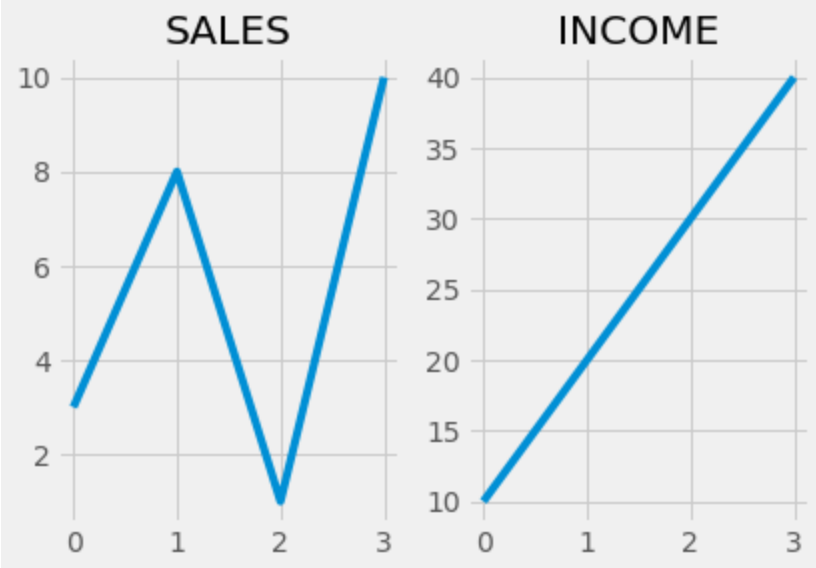
Title

You can add a title to each plot with the title() function:

EXAMPLE: 2 plots, with titles

In [40]:

```
import matplotlib.pyplot as plt
import numpy as np

#plot 1
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1,2,1)
plt.plot(x,y)
plt.title("SALES")

#plot 2
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1,2,2)
plt.plot(x,y)
plt.title("INCOME")

plt.show()
```



Super Title

You can add a title to the entire figure with the suptitle() function:

EXAMPLE: Add a title for the entire figure

In [41]:

```
import matplotlib.pyplot as plt
import numpy as np

#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)
plt.plot(x,y)
plt.title("SALES")

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)
plt.title("INCOME")

plt.suptitle("MY SHOP")
plt.show()
```

Scatter:

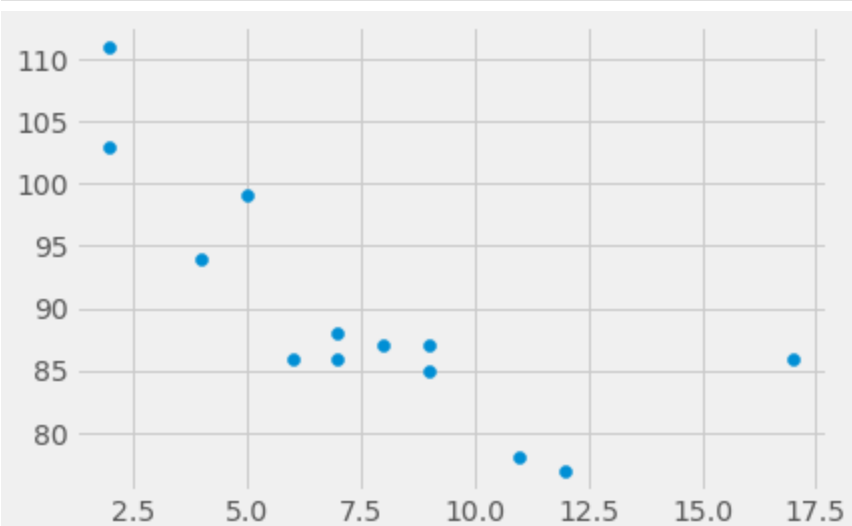
Creating Scatter Plots

With Pyplot, you can use the scatter() function to draw a scatter plot.

The scatter() function plots one dot for each observation. It needs two arrays of the same length, one for the values of the x-axis, and one for values on the y-axis:

EXAMPLE: A simple scatter plot

In [42]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x,y)
plt.show()
```



The observation in the example above is the result of 13 cars passing by.

The X-axis shows how old the car is.

The Y-axis shows the speed of the car when it passes.

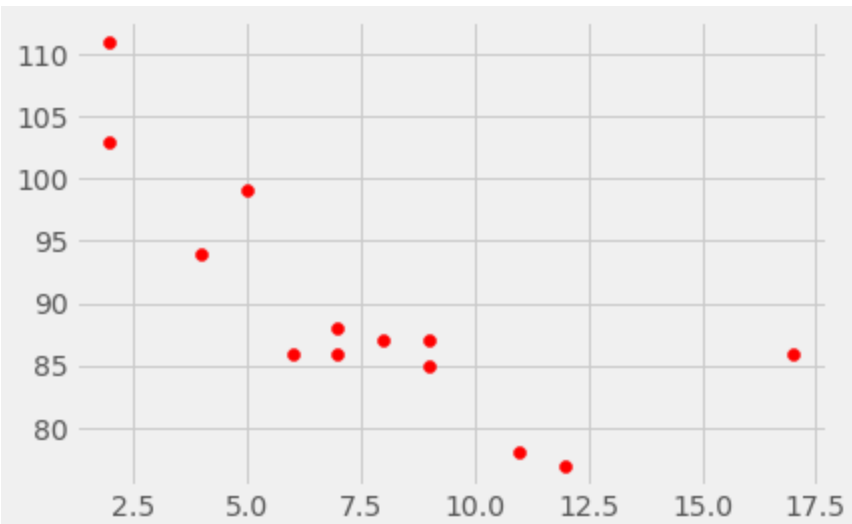
Are there any relationships between the observations?

It seems that the newer the car, the faster it drives, but that could be a coincidence, after all we only registered 13 cars.

Color

You can change the color of dots.

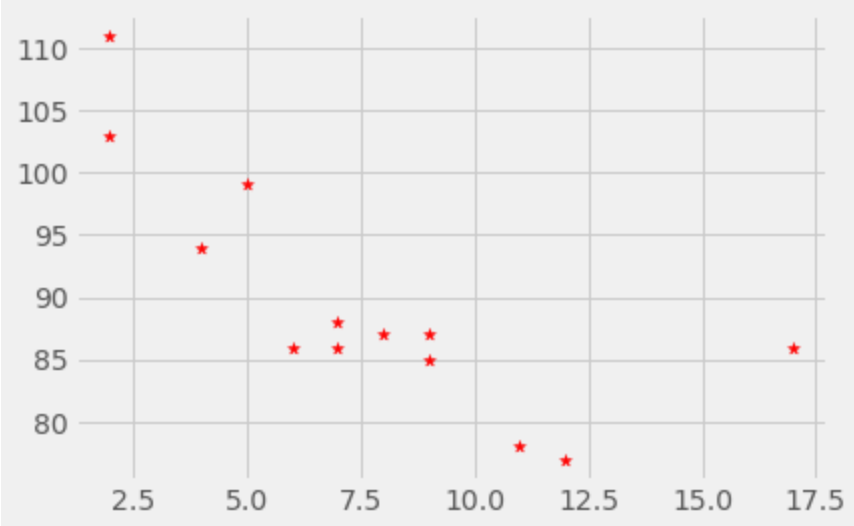
In [21]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x,y, c= 'r' )
plt.show()
```



Change the design of the marker

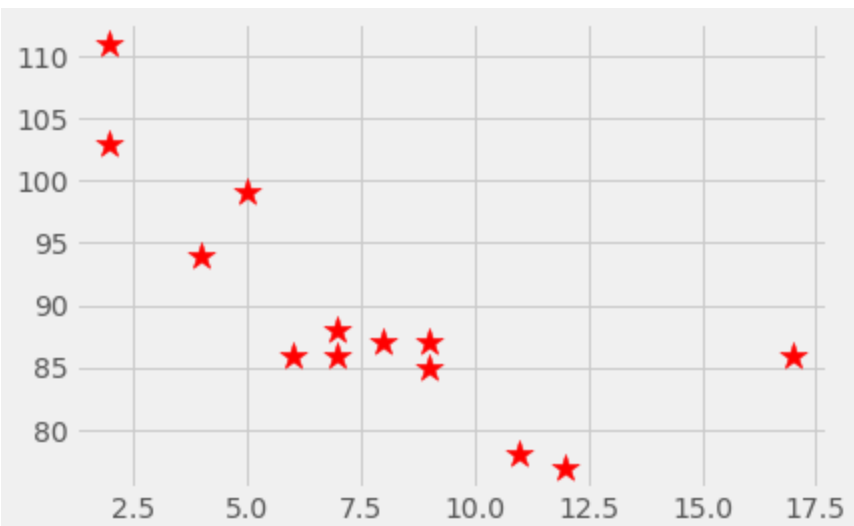
- **You can change the design of the marker by using the merker parameter.**

In [28]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x,y, c= 'r',marker = "*" )
plt.show()
```



Change the size of the marker.

In [31]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x,y, c= 'r',marker = "*", s = 200)
plt.show()
```



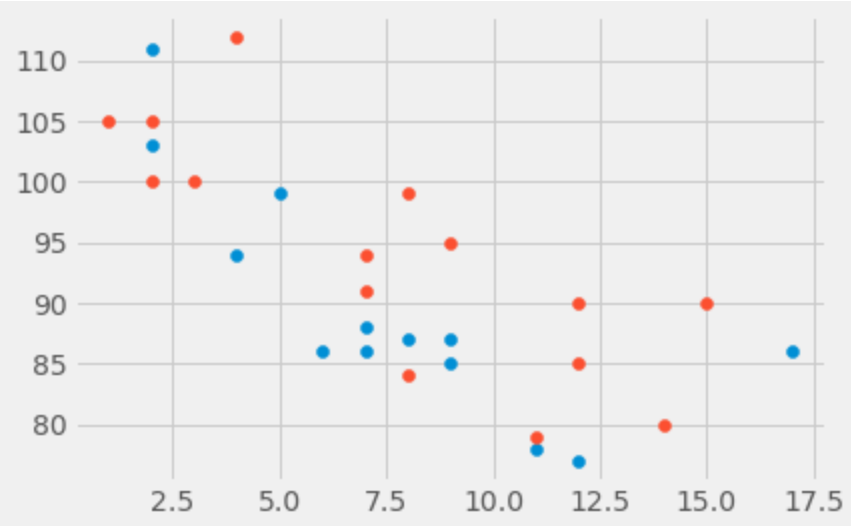
Compare Plots

In the example above, there seems to be a relationship between speed and age, but what if we plot the observations from another day as well? Will the scatter plot tell us something else?

EXAMPLE: Draw two plots on the same figure

In [43]:
```
import matplotlib.pyplot as plt
import numpy as np

#day one, the age and speed of 13 cars:
x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y)
```

```
#day two, the age and speed of 15 cars:
x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
plt.scatter(x, y)

plt.show()
```

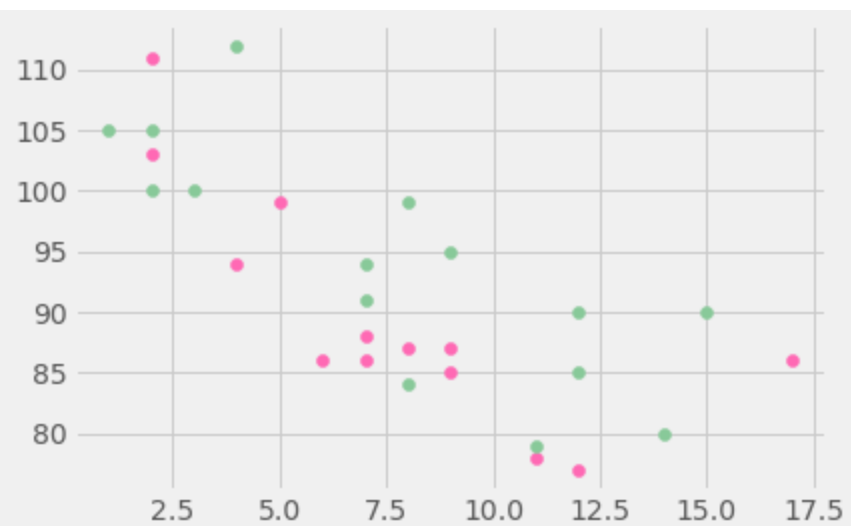


By comparing the two plots, I think it is safe to say that they both gives us the same conclusion: the newer the car, the faster it drives.

Colors You can set your own color for each scatter plot with the color or the c argument:

EXAMPLE: Set your own color of the markers

In [44]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y, color = 'hotpink')

x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
plt.scatter(x, y, color = '#88c999')

plt.show()
```

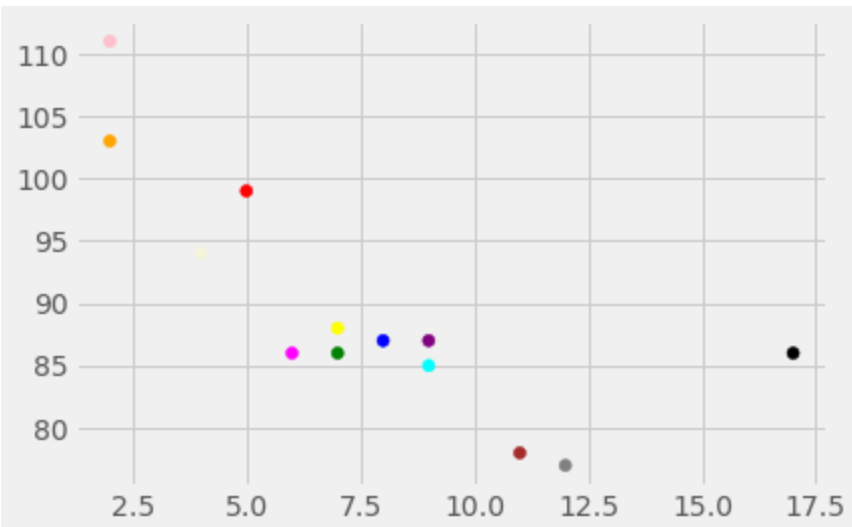


Color Each Dot

You can even set a specific color for each dot by using an array of colors as value for the c argument:

Example: Set your own color of the markers

In [45]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors = np.array(["red","green","blue","yellow","pink","black","orange","purple","beige"

plt.scatter(x, y, c=colors)

plt.show()
```

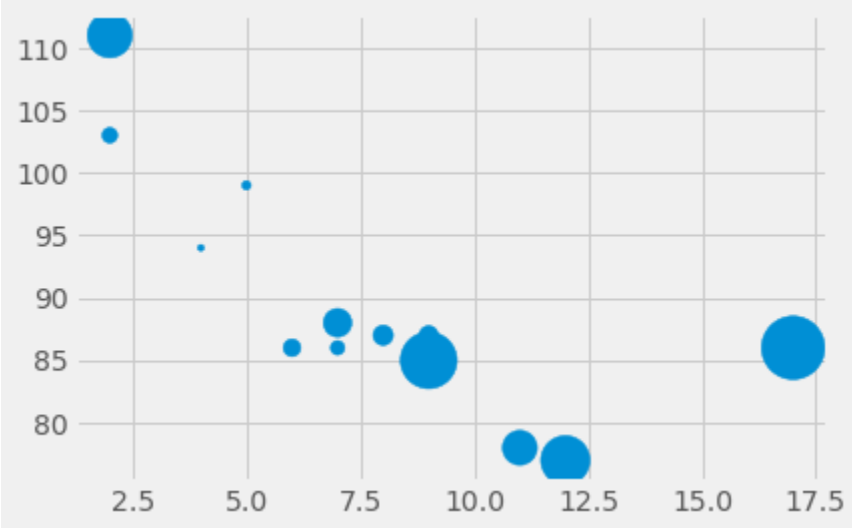


Size

You can change the size of the dots with the s argument.

Just like colors, make sure the array for sizes has the same length as the arrays for the x- and y-axis:

In [10]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
sizes = np.array([20,50,100,200,500,1000,60,90,10,300,600,800,75])

plt.scatter(x,y, s= sizes)

plt.show()
```



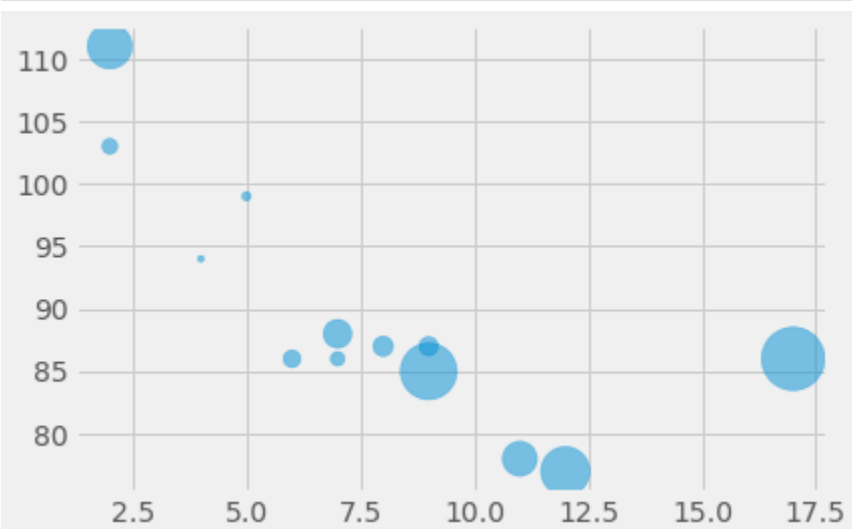
Alpha

You can adjust the transparency of the dots with the alpha argument.

Just like colors, make sure the array for sizes has the same length as the arrays for the x- and y-axis:

EXAMPLE:Set your own size for the markers

In [11]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
sizes = np.array([20,50,100,200,500,1000,60,90,10,300,600,800,75])

plt.scatter(x, y, s=sizes, alpha=0.5)

plt.show()
```



You can use others function also for scatter plot that we used in line plot like title(), xlabel()

Bars:

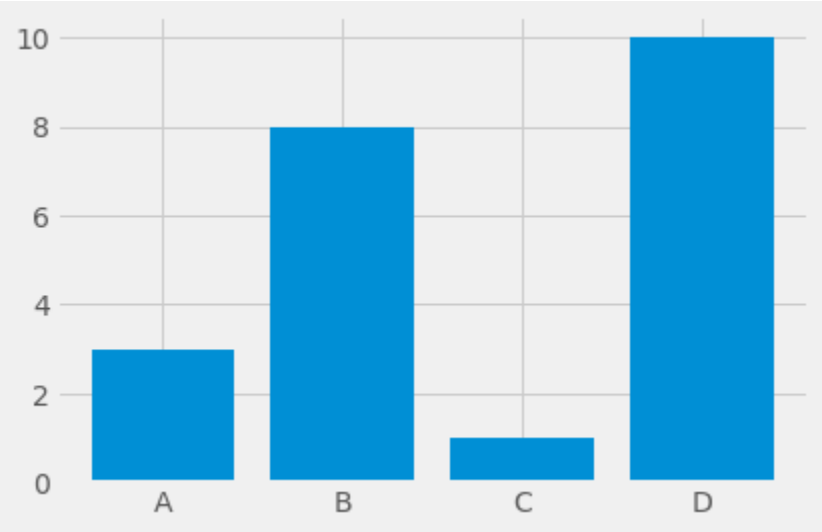
Creating Bars

With Pyplot, you can use the bar() function to draw bar graphs:

EXAMPLE:Draw 4 bars

In [46]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
```

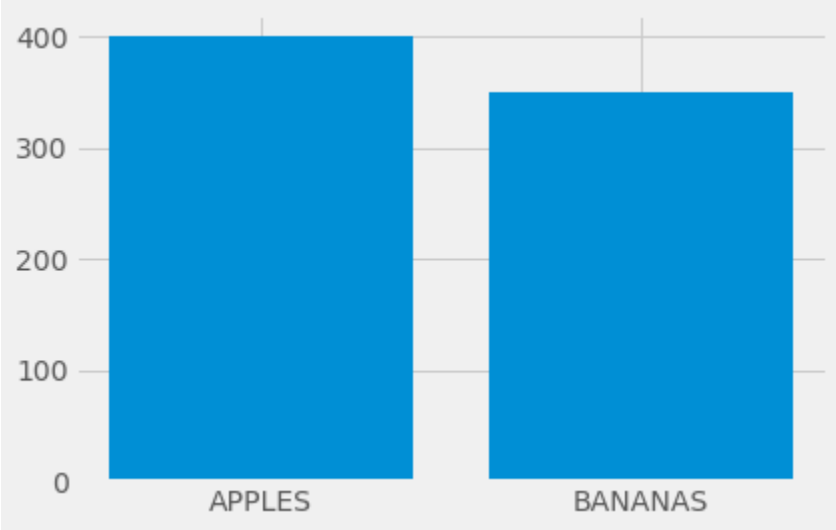
Out[46]:
```
<BarContainer object of 4 artists>
```



The bar() function takes arguments that describes the layout of the bars.

The categories and their values represented by the first and second argument as arrays.

In [47]:
```
import matplotlib.pyplot as plt
import numpy as np

x = ["APPLES", "BANANAS"]
y = [400, 350]

plt.bar(x,y)
plt.show()
```

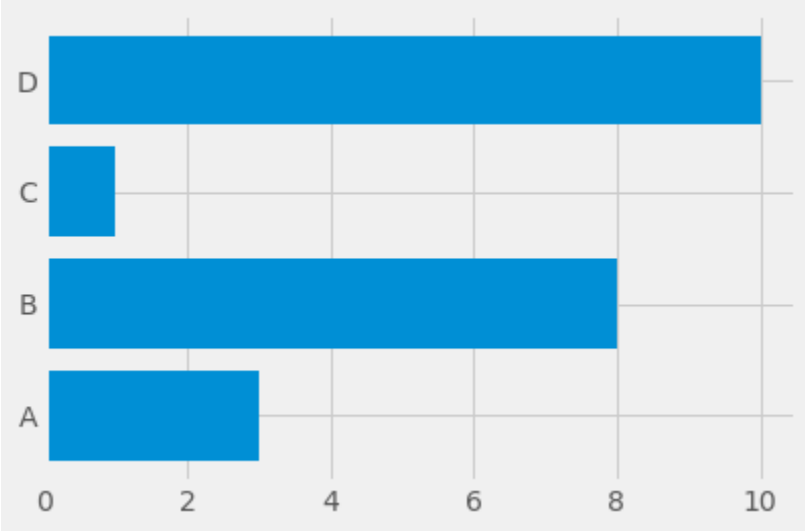


Horizontal Bars

If you want the bars to be displayed horizontally instead of vertically, use the barh() function:

EXAMPLE: Draw 4 horizontal bars

In [48]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x,y)
plt.show()
```



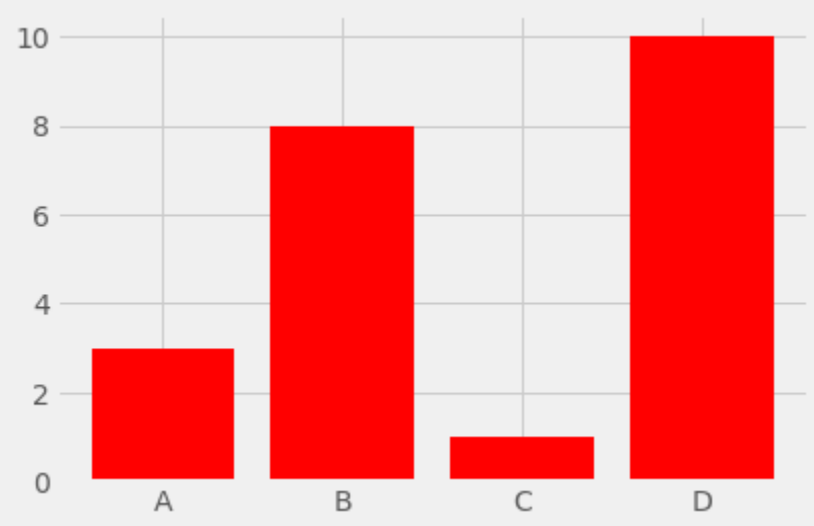
Bar Color

The bar() and barh() takes the keyword argument color to set the color of the bars:

EXAMPLE: Draw 4 red bars

In [49]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(['A','B','C','D'])
```

```
y = np.array([3,8,1,10])

plt.bar(x,y, color ='red')
plt.show()
```



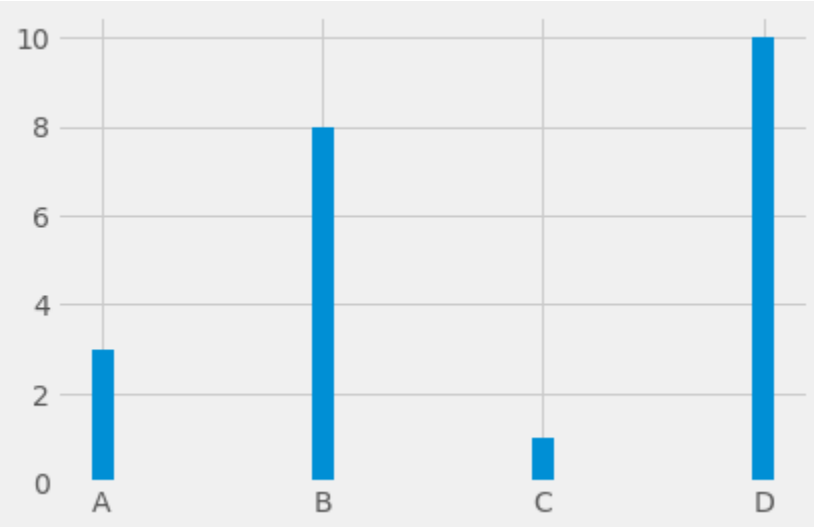
You can use different colous and Color Hex also.

Bar Width

The bar() takes the keyword argument width to set the width of the bars:

EXAMPLE: Draw 4 very thin bars

In [50]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y, width = 0.1)
plt.show()
```



The default width value is 0.8

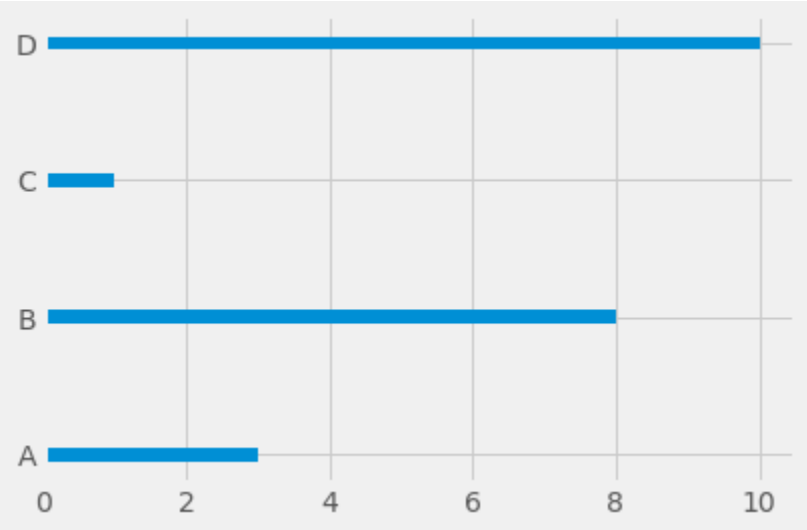
For horizontal bars, use height instead of width.

Bar Height

The barh() takes the keyword argument height to set the height of the bars:

EXAMPLE: Draw 4 very thin bars

In [51]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x, y, height = 0.1)
plt.show()
```

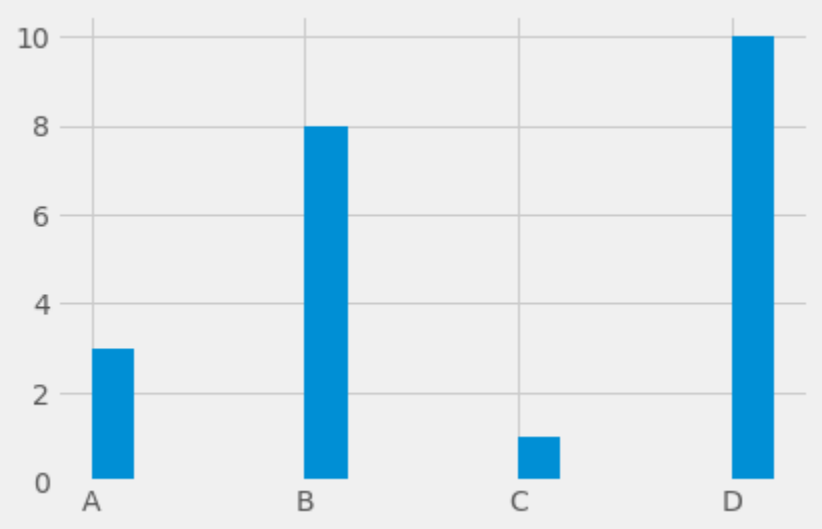


The default height value is 0.8

Bar edge

You can chnage position of the bar by using align parameter.

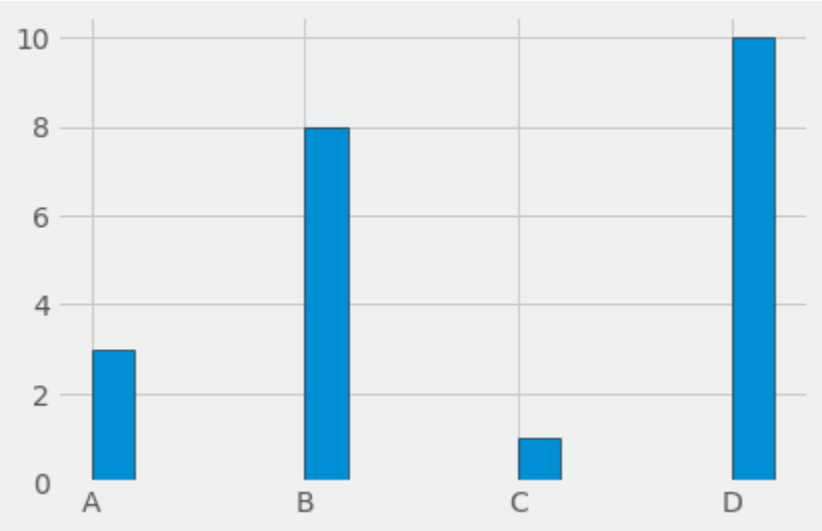
In [52]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y, width = 0.2, align = 'edge')
plt.show()
```



Edge color

You can change the color of the edges by using edgecolor function

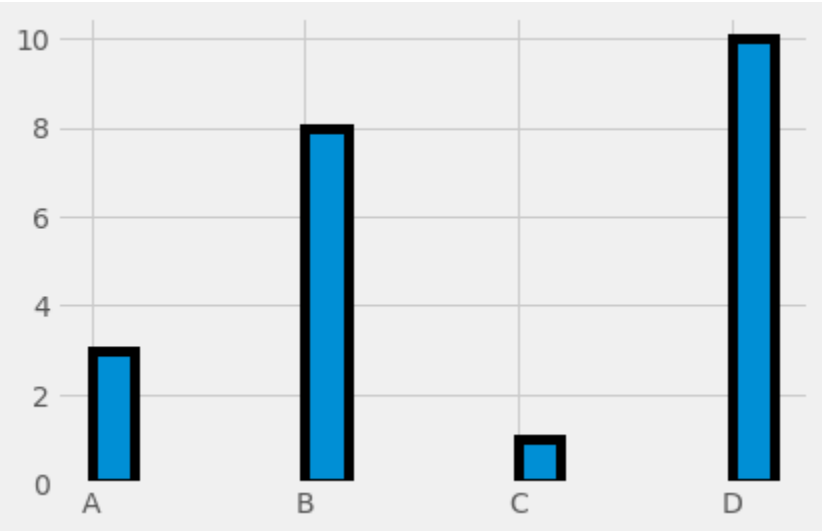
In [53]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y, width = 0.2,align = 'edge',edgecolor='k')
plt.show()
```



Edge width

You can chnage the widtth of edge by using linewidth parameter.

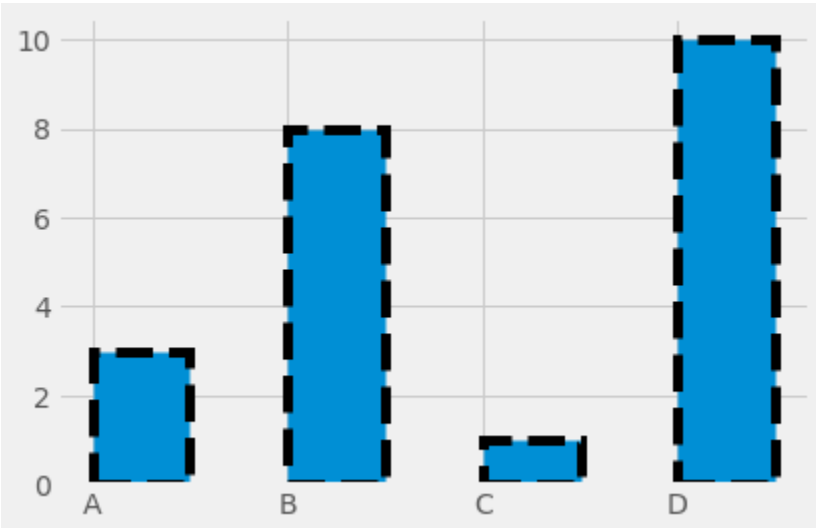
In [54]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y, width = 0.2,align = 'edge',edgecolor='k',linewidth = 5)
plt.show()
```



Edge Line Style

You can chnage the style of edge line by using linestyle parameter.

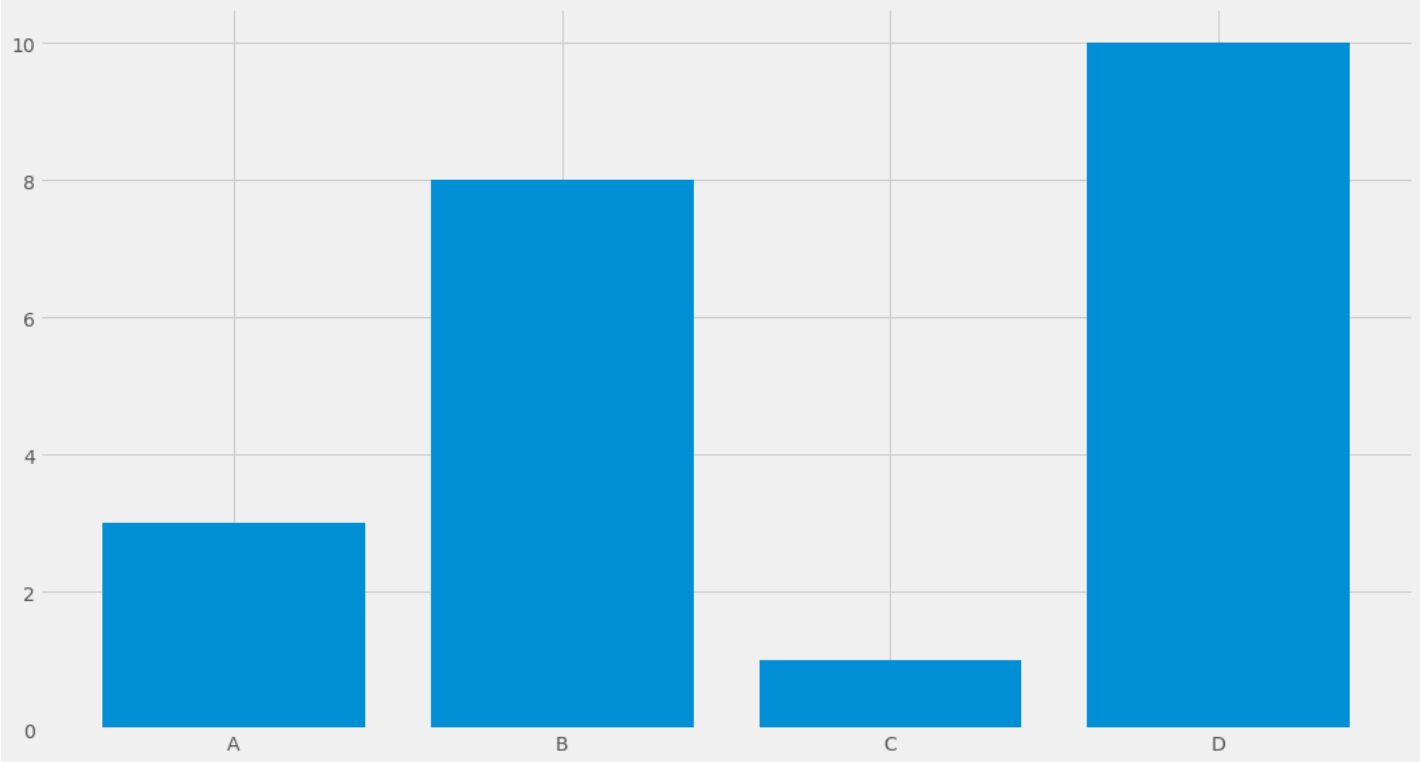
In [55]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y, width = 0.5,align = 'edge',edgecolor='k',linewidth = 5,linestyle='--')
plt.show()
```



Change the figure size

- **To change the size of the figure we have figure() function.**

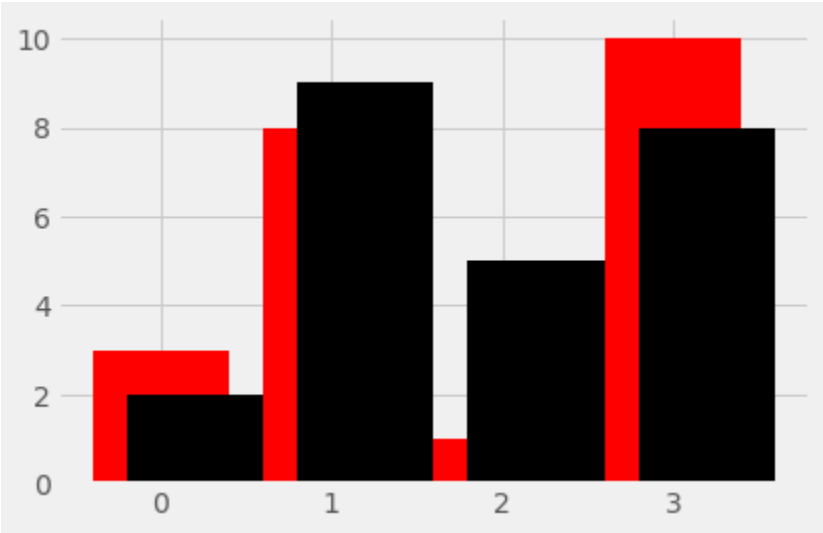
In [20]:
```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(16,9))

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
plt.show()
```



More than One data sets

Sometimes we have two or more sets to represent. For that you have to just use another bar() function

In [82]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
z = np.array([2, 9, 5, 8])

x_index = np.arange(len(x))


width = 0.2

plt.bar(x_index,y, color ='r')
plt.bar(x_index + width ,z,color ='k')

plt.show()
```

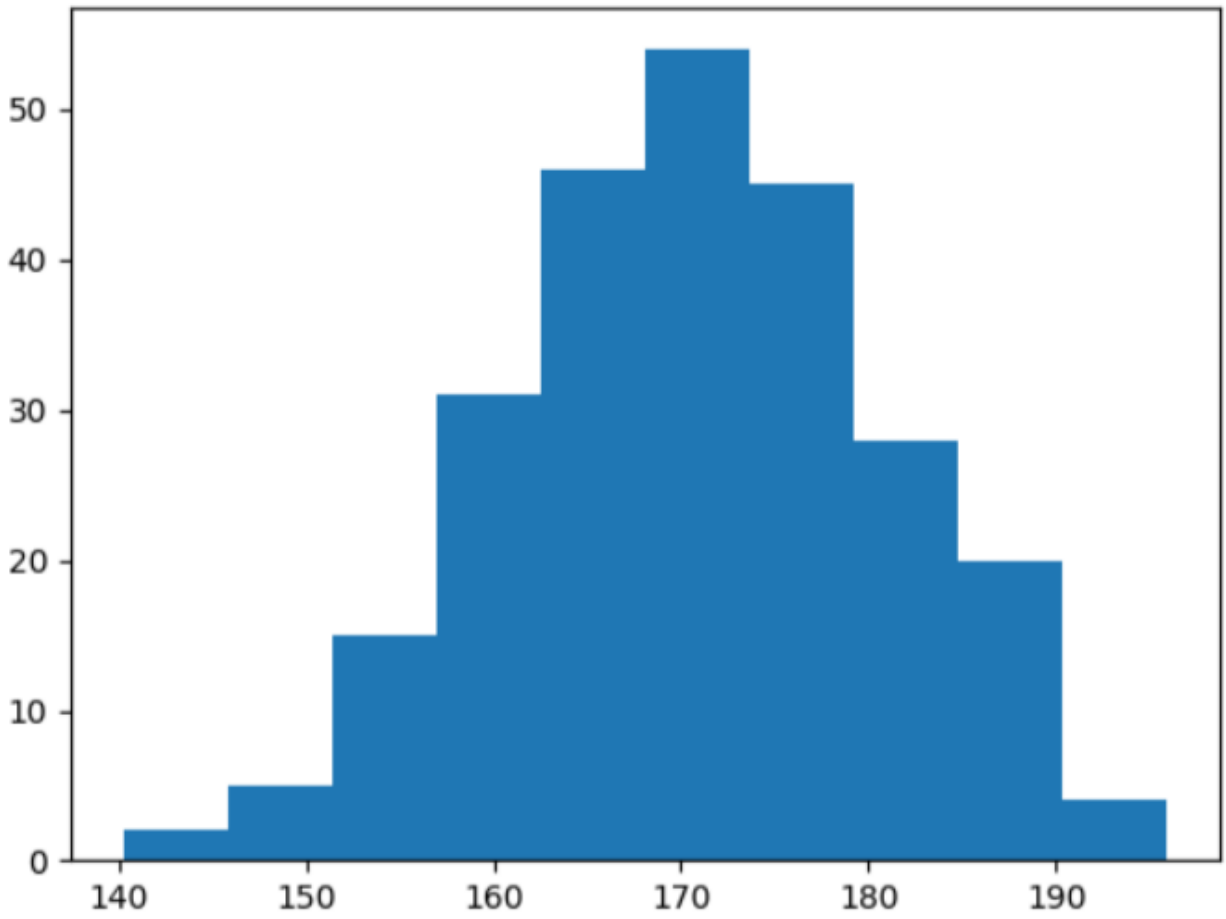


But it overlap each ohter, we cant't get result properly.

Histogram:

- **A histogram is a graph showing frequency distributions.**
- **It is a graph showing the number of observations within each given interval.**

EXAMPLE: Say you ask for the height of 250 people, you might end up with a histogram like this:



You can read from the histogram that there are approximately:

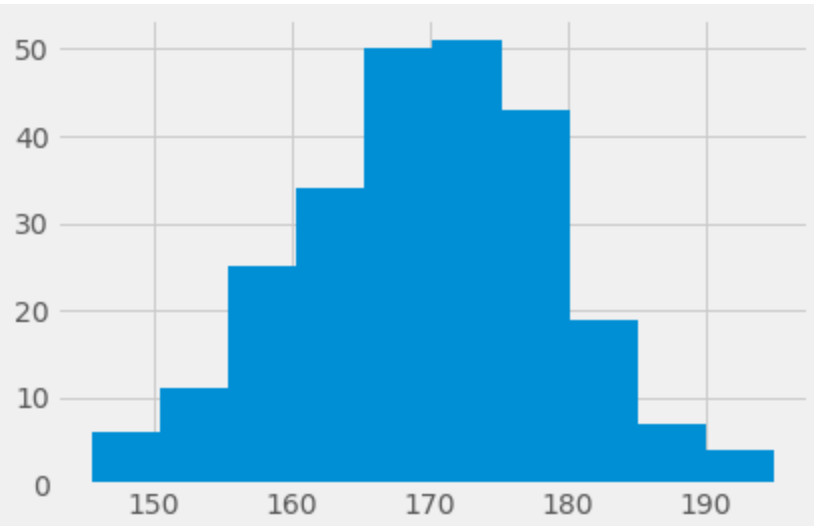
2 people from 140 to 145cm
5 people from 145 to 150cm
15 people from 151 to 156cm
31 people from 157 to 162cm
46 people from 163 to 168cm
53 people from 168 to 173cm
45 people from 173 to 178cm
28 people from 179 to 184cm
21 people from 185 to 190cm
4 people from 190 to 195cm

Create Histogram

- **In Matplotlib, we use the hist() function to create histograms.**
- **The hist() function will use an array of numbers to create a histogram, the array is sent into the function as an argument.**
- **For simplicity we use NumPy to randomly generate an array with 250 values, where the values will concentrate around 170, and the standard deviation is 10.**

EXAMPLE: A simple histogram

In [56]:
```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(170, 10, 250)

plt.hist(x)
plt.show()
```



let's take another example

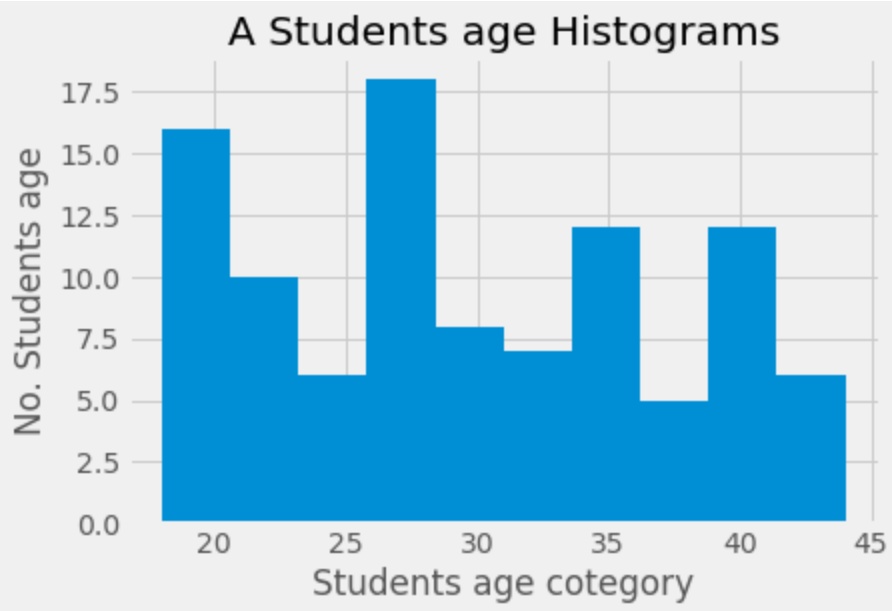
First create the data or information

In [57]:
```
import numpy as np
import random

a_student_age = np.random.randint(18,45,(100))
b_student_age = np.random.randint(15,40,(100))

print(a_student_age,'\n')

print(b_student_age)
```

```
[20 23 44 19 28 30 24 33 38 27 35 40 28 32 36 28 36 33 29 19 23 28 39 28
 30 28 21 32 18 32 34 20 19 26 19 39 32 25 42 19 19 34 28 36 39 36 27 20
 30 26 38 35 31 20 19 24 37 26 42 35 20 40 29 27 39 37 43 40 40 27 23 22
 25 26 29 20 22 34 35 29 25 21 27 26 41 29 39 41 25 18 44 23 26 19 37 42
 22 35 23 40]

[15 39 33 38 38 38 20 36 32 34 15 30 17 28 39 36 15 22 34 24 37 34 39 34
 19 18 38 30 22 38 17 26 29 22 33 31 27 18 19 32 28 38 30 15 30 20 21 37
 28 18 15 39 20 21 15 23 26 28 15 32 27 15 28 25 23 34 35 39 22 27 32 25
 20 35 32 16 28 17 18 22 27 18 19 17 22 19 27 37 28 31 33 23 22 20 28 27
 25 18 21 31]
```

In [58]:
```
plt.hist(a_student_age)
```

```
plt.title("A Students age Histograms")
plt.xlabel("Students age cotegory")
plt.ylabel("No. Students age")

plt.show()
```



Histogram Parameters:

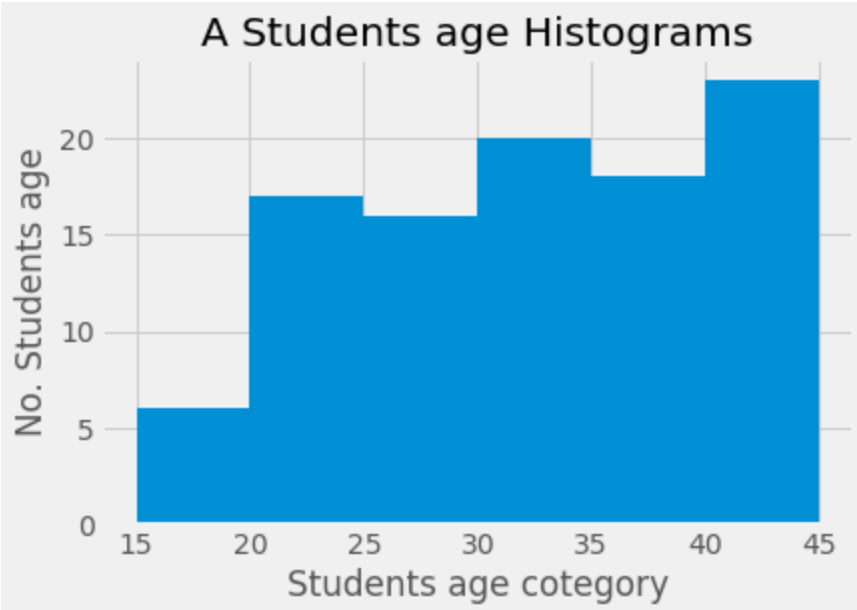
bins=None,
range=None,
density=False,
weights=None,
cumulative=False,
bottom=None,
histtype='bar',
align='mid',
orientation='vertical',
rwidth=None,
log=False,
color=None,
label=None,
stacked=False,
***,**
data=None,
****kwargs,**

bins:

- **This parameter is used to set the range of the bars.**
- **arguments:{int or sequence or str, default: :rc: `hist.bins` }**

In [17]:
```
import matplotlib.pyplot as plt
import numpy as np

a_student_age = np.random.randint(18,45,(100))
```

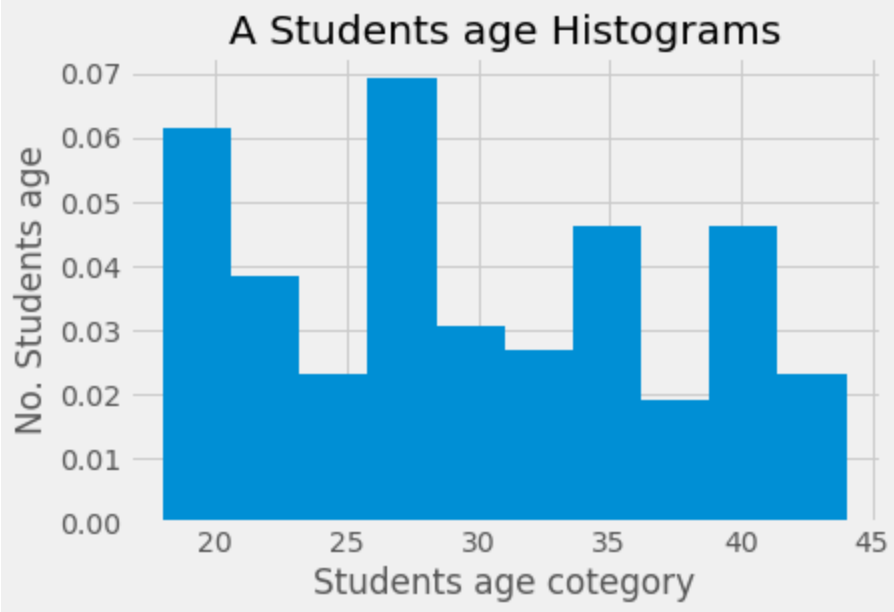
```
# To set the range of bar
bins = [15,20,25,30,35,40,45] # we set from 15 to 45 because our data is start from 15 an


plt.hist(a_student_age,bins)

plt.title("A Students age Histograms")
plt.xlabel("Students age cotegory")
plt.ylabel("No. Students age")

plt.show()
```



density:

draw and return a probability density: each bin will display the bin's raw count divided by the total number ofcounts *and the bin width*

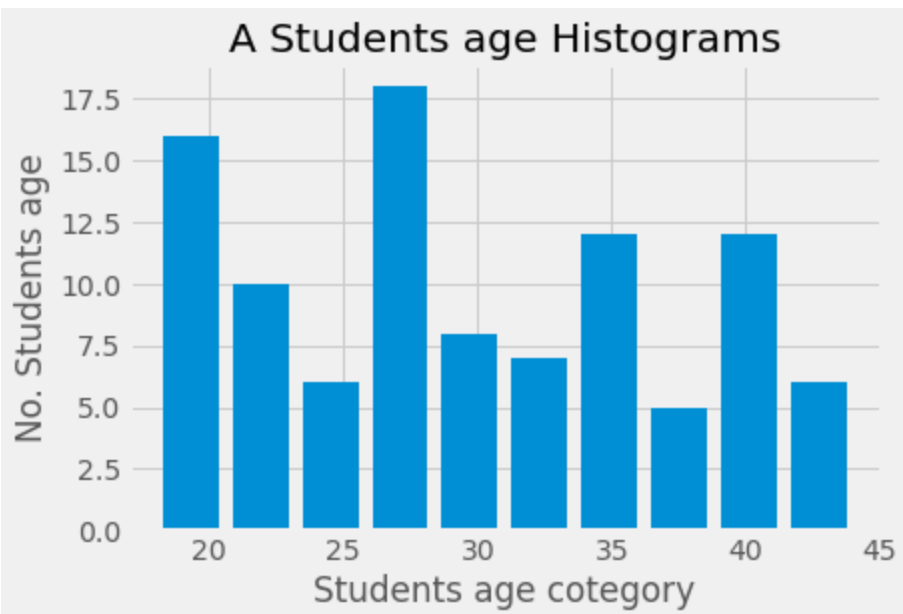
In [60]:
```
plt.hist(a_student_age,density= True)

plt.title("A Students age Histograms")
plt.xlabel("Students age cotegory")
plt.ylabel("No. Students age")

plt.show()
```



rwidth:

To set the width of the bar, by default it is 1

In [61]:
```
plt.hist(a_student_age,rwidth = 0.8,)

plt.title("A Students age Histograms")
plt.xlabel("Students age cotegory")
plt.ylabel("No. Students age")

plt.show()
```



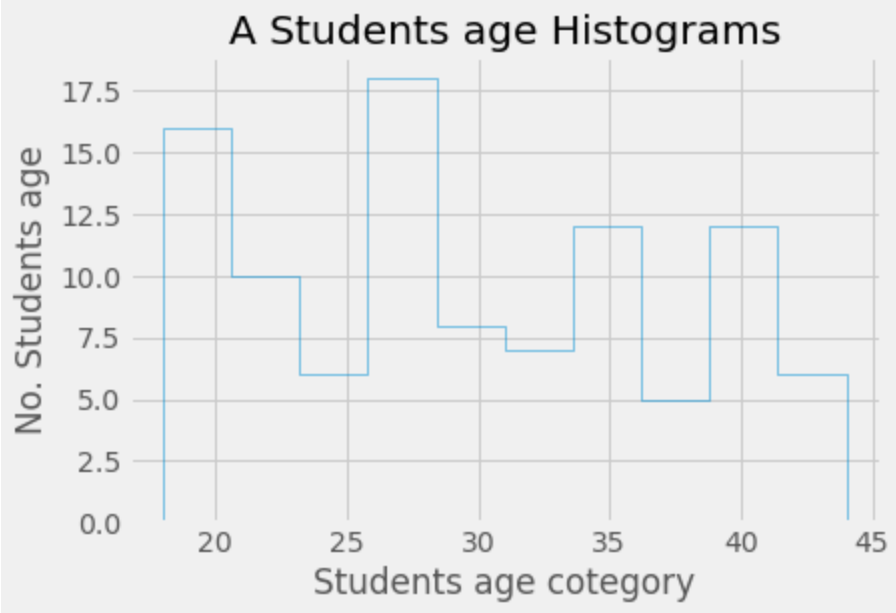
histtype:

It changes the type of histogram

It has four arguments {'bar','barstacked','step','stepfilled'}

In [62]:
```
plt.hist(a_student_age,histtype = 'step')

plt.title("A Students age Histograms")
plt.xlabel("Students age cotegory")
plt.ylabel("No. Students age")
```

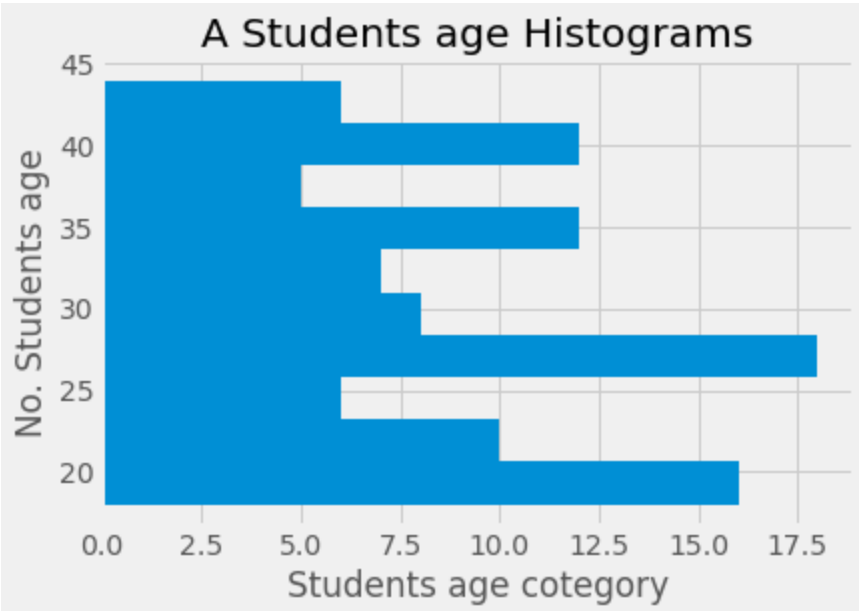
```
plt.show()
```



orientation:

To make graph horizonatl or vertical
arguments:{'vertical','horizontal}
By default it is vertical

In [63]:
```
plt.hist(a_student_age, orientation = 'horizontal')

plt.title("A Students age Histograms")
plt.xlabel("Students age cotegory")
plt.ylabel("No. Students age")

plt.show()
```

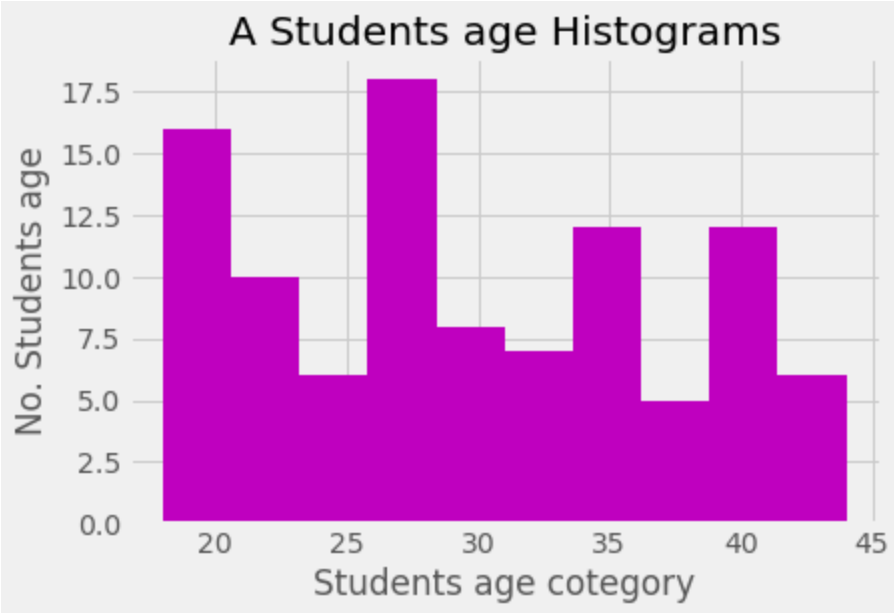


color:

To set the colour of the bar.

In [64]:
```
plt.hist(a_student_age, color = 'm')
```

```
plt.title("A Students age Histograms")
plt.xlabel("Students age cotegory")
plt.ylabel("No. Students age")

plt.show()
```



Pie Charts:

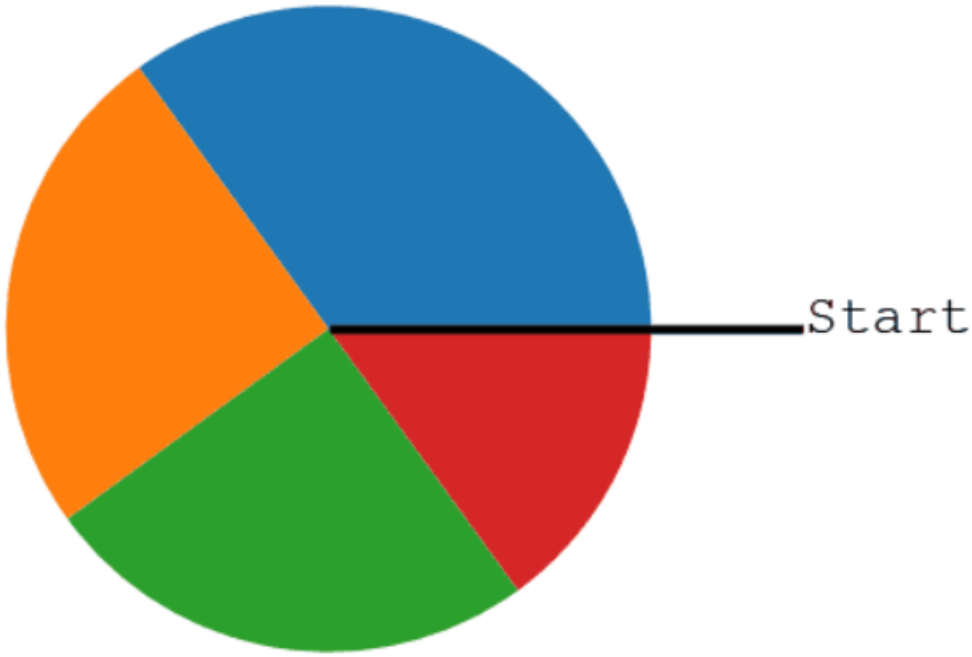
Creating Pie Charts With Pyplot, you can use the pie() function to draw pie charts:

EXAMPLE: A simple pie chart

In [12]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])

plt.pie(y)
plt.show()
```

As you can see the pie chart draws one piece (called a wedge) for each value in the array (in this case [35, 25, 25, 15]).

By default the plotting of the first wedge starts from the x-axis and move counterclockwise:



The size of each wedge is determined by comparing the value with all the other values, by using this formula:

The value divided by the sum of all values: x/sum(x)

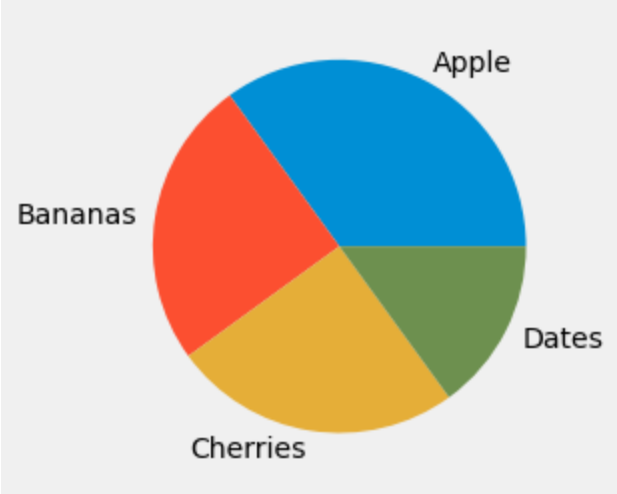
Labels

Add labels to the pie chart with the label parameter.

The label parameter must be an array with one label for each wedge:

EXAMPLE: A simple pie chart

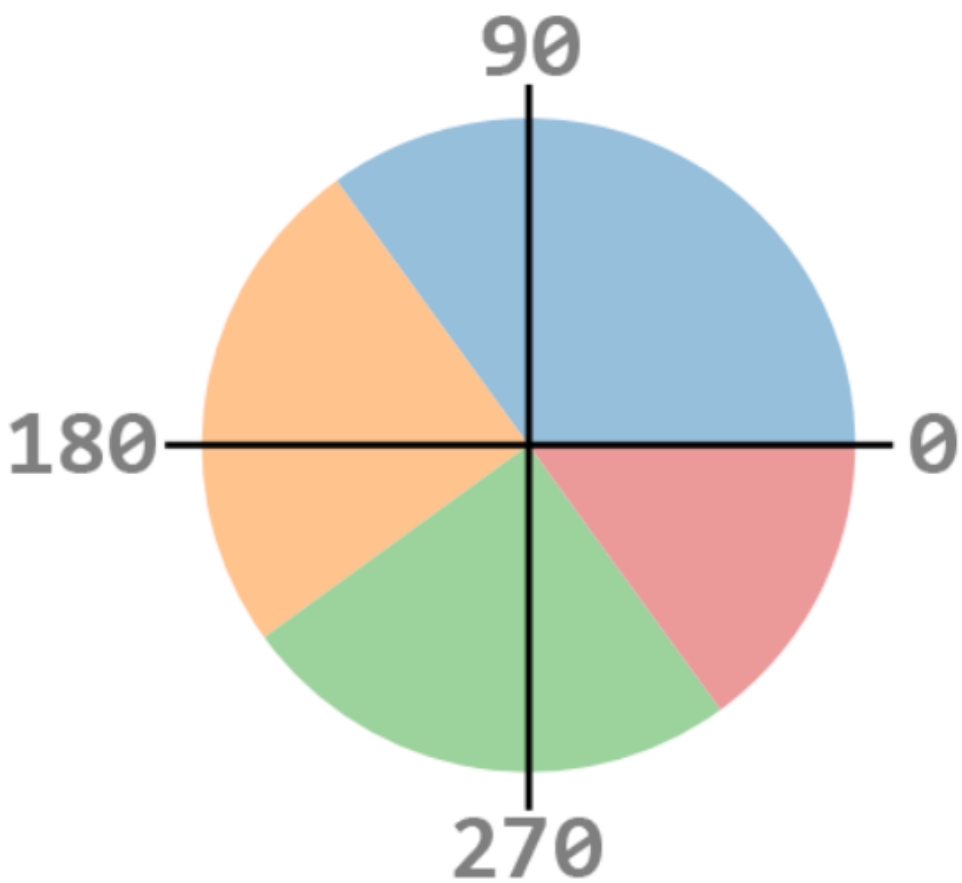
In [33]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apple", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.show()
```



Start Angle

As mentioned the default start angle is at the x-axis, but you can change the start angle by specifying a lstartangle parameter.

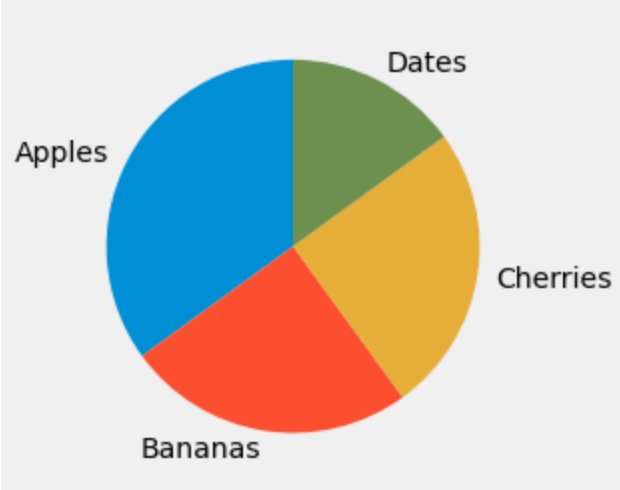
The lstartangle parameter is defined with an angle in degrees, default angle is 0:



EXAMPLE:Start the first wedge at 90 degrees

In [67]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
```

```
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels, startangle = 90)
plt.show()
```



Explode

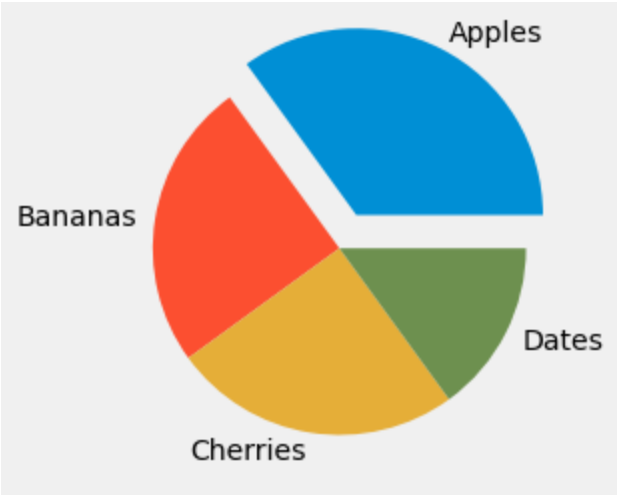
Maybe you want one of the wedges to stand out? The explode parameter allows you to do that.

The explode parameter, if specified, and not None, must be an array with one value for each wedge.

Each value represents how far from the center each wedge is displayed:

Example:Pull the "Apples" wedge 0.2 from the center of the pie

In [68]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode)
plt.show()
```

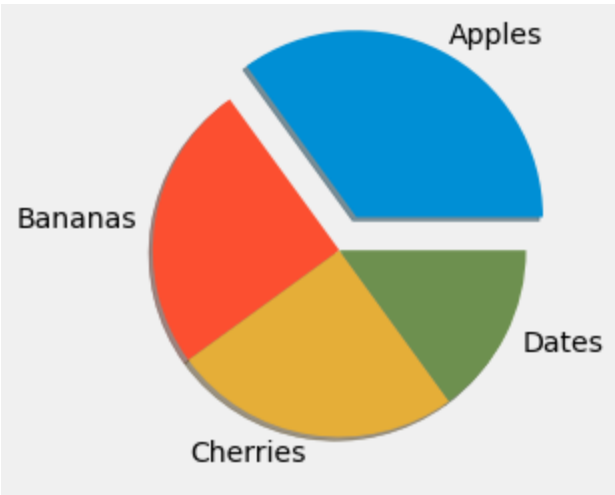


Shadow

Add a shadow to the pie chart by setting the shadows parameter to True:

Example:Add a shadow

In [40]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode, shadow = True)
plt.show()
```



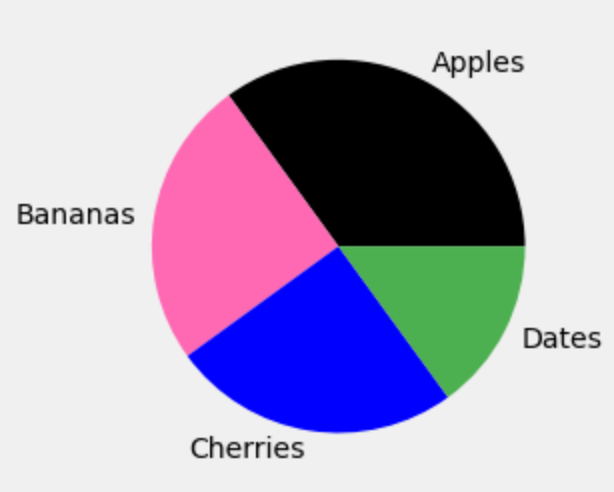
Colors

You can set the color of each wedge with the colors parameter.

The colors parameter, if specified, must be an array with one value for each wedge:

Example: Specify a new color for each wedge

In [70]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
mycolors = ["black", "hotpink", "b", "#4CAF50"]

plt.pie(y, labels = mylabels, colors = mycolors)
plt.show()
```

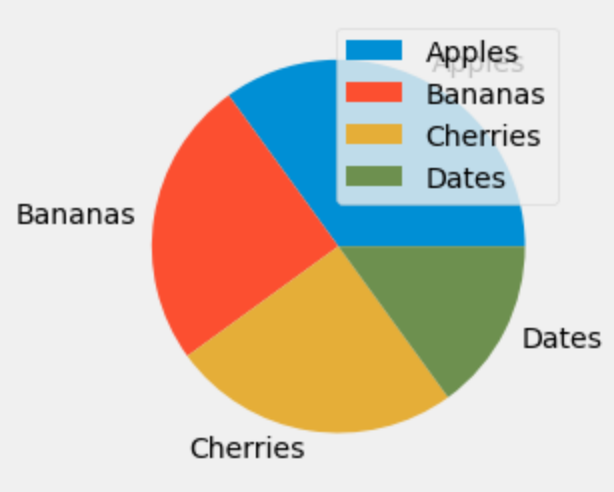


Legend

To add a list of explanation for each wedge, use the legend() function:

EXAMPLE: Add a legend

In [71]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.legend()
plt.show()
```



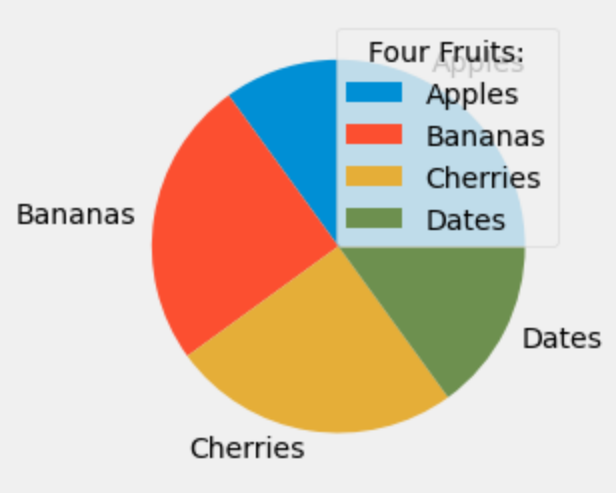
Legend With Header

To add a header to the legend, add the title parameter to the legend function.

EXAMPLE: Add a legend with a header

In [72]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
```

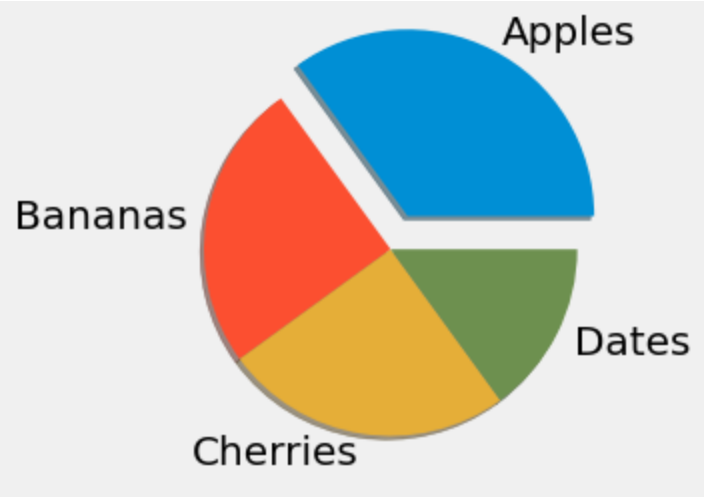
```
plt.legend(title = "Four Fruits:")
plt.show()
```



To change the size of the Text

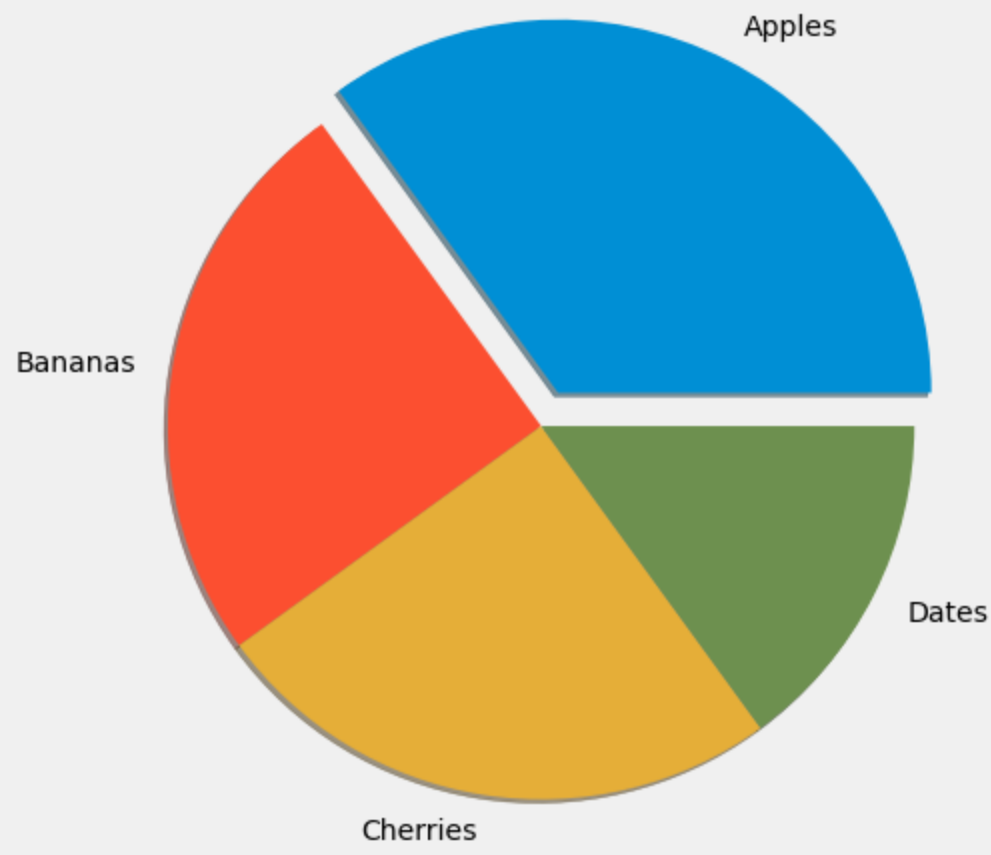
To change the size of the text we have the textprops parameter.

In [39]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

text_properties = {"fontsize":20}

plt.pie(y, labels = mylabels, explode = myexplode, shadow = True, textprops = text_proper
plt.show()
```



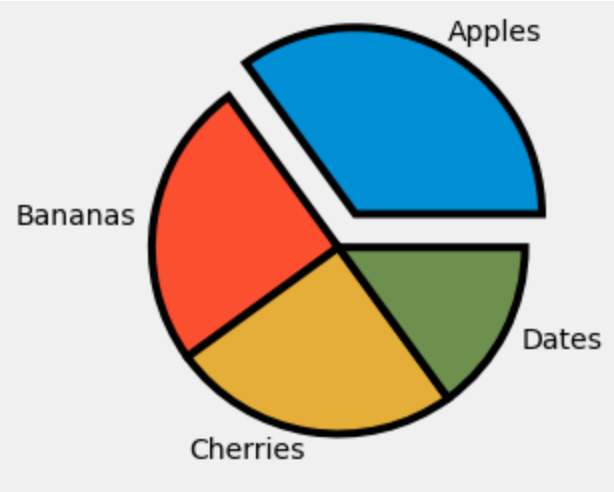
To change the size/radious of pie chart

In [42]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode, shadow = True,radius = 2 )
plt.show()
```



To change the wedge properties

In [46]:
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

wedge_properties = {"linewidth":4, "width":1, "edgecolor":"k"}

plt.pie(y, labels = mylabels, explode = myexplode,wedgeprops = wedge_properties )
plt.show()
```

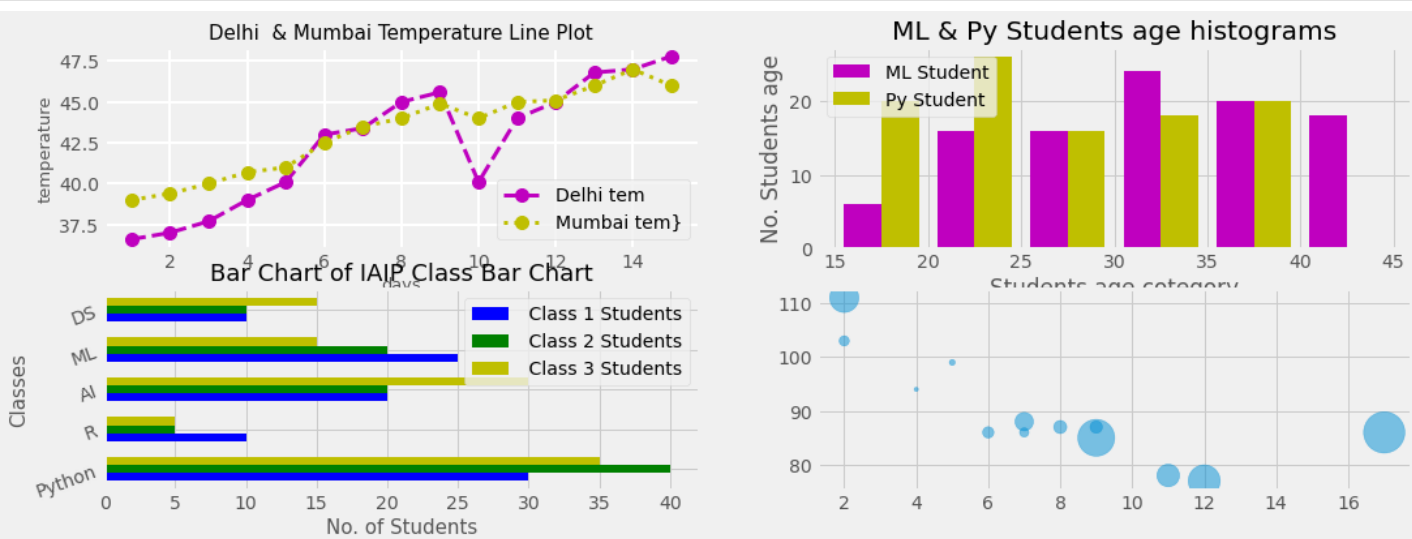


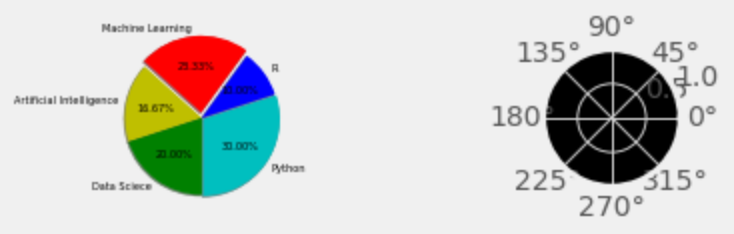
Let's plot all graph by using subplot

In [55]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pda
```

```
plt.figure(figsize=(16,9))

##----------------------------------------start
#plt.subplot(3,2,1)
plt.subplot(321)
#*********************************************Line Plot
days = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
delhi_tem = [36.6, 37, 37.7,39,40.1,43,43.4,45,45.6,40.1,44,45,46.8,47,47.8]
mumbai_tem = [39,39.4,40,40.7,41,42.5,43.5,44,44.9,44,45,45.1,46,47,46]

plt.plot(days, delhi_tem, "mo--", linewidth = 3,
        markersize = 10, label = "Delhi tem")

plt.plot(days, mumbai_tem, "yo:", linewidth = 3,
        markersize = 10, label = "Mumbai tem}")

plt.title("Delhi  & Mumbai Temperature Line Plot", fontsize=15)
plt.xlabel("days",fontsize=13)
plt.ylabel("temperature",fontsize=13)
plt.legend(loc = 4)
plt.grid(color='w', linestyle='-', linewidth=2)

#-------------------------------------------------------------end

plt.subplot(3,2,2) ##-------------------------------------------------start
#*****************************************************************histograms
ml_students_age = np.random.randint(18,45, (100))
py_students_age = np.random.randint(15,40, (100))
bins = [15,20,25,30,35,40,45]

plt.hist([ml_students_age, py_students_age], bins, rwidth=0.8, histtype = "bar",
         orientation='vertical', color = ["m", "y"], label = ["ML Student", "Py Student"]

plt.title("ML & Py Students age histograms")
plt.xlabel("Students age cotegory")
plt.ylabel("No. Students age")
plt.legend()
#-------------------------------------------------------------------end

plt.subplot(3,2,3) ##-------------------------------------------start
#*************************************************************Bar Chart
classes = ["Python", "R", "AI", "ML", "DS"]
class1_students = [30, 10, 20, 25, 10] # out of 100 student in each class
class2_students = [40, 5, 20, 20, 10]
class3_students = [35, 5, 30, 15, 15]
classes_index = np.arange(len(classes))

width = 0.2

plt.barh(classes_index, class1_students, width , color = "b",
        label =" Class 1 Students") #visible=False

plt.barh(classes_index + width, class2_students, width , color = "g",
        label =" Class 2 Students")

plt.barh(classes_index + width + width, class3_students, width , color = "y",
        label =" Class 3 Students")

plt.yticks(classes_index + width, classes, rotation = 20)
plt.title("Bar Chart of IAIP Class Bar Chart", fontsize = 18)
plt.ylabel("Classes",fontsize = 15)
plt.xlabel("No. of Students", fontsize = 15)
```

```
plt.legend()
#-------------------------------------------------------------------end

plt.subplot(3,2,4) ##-----------------------------------------------start
#*****************************************************************Scatter Plot
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
sizes = np.array([20,50,100,200,500,1000,60,90,10,300,600,800,75])

plt.scatter(x, y, s=sizes, alpha=0.5)

plt.show()
#-----------------------------------------------------------------------end


plt.subplot(3,2,5) ##---------------------------------------------start
#*****************************************************************Pie plot
classes = ["Python", 'R', 'Machine Learning', 'Artificial Intelligence',
           'Data Sciece']
class1_students = [45, 15, 35, 25, 30]
explode = [0.03,0,0.1,0,0]
colors = ["c", 'b','r','y','g']
textprops = {"fontsize":5}

plt.pie(class1_students,
        labels = classes,
        explode = explode,
        colors =colors,
        autopct = "%0.2f%%",
        shadow = True,
        radius = 1.4,
       startangle = 270,
        textprops =textprops)
#-------------------------------------------------------------end


plt.subplot(3,2,6, projection='polar', facecolor='k' ,frameon=True)

plt.show()
```




Machine Learning
29.33%
R
10.00%
Artificial Intelligence
16.67%
20.00%
33.00%
Python
Data Science
90°
135°
45°
1.0
180°
0°
225
315°
270°

In [ ]: